

Advanced Java Programming LAB

(CIE-306P)

Faculty Name : Dr. Ashish Khanna

Student Name : Yash Pandey

Roll No : 12214802722 (AIML II-B)

Semester : 6th Semester



Maharaja Agrasen Institute of Technology, PSP Area,
Sector – 22, Rohini, New Delhi - 110086



MAHARAJA AGRASEN INSTITUTE OF TECHNOLOGY

VISION

To attain global excellence through education, innovation, research, and work ethics with the commitment to serve humanity digitize educational resources and process for enhanced teaching and effective learning.

MISSION

- M1. To promote diversification by adopting advancement in science, technology, management, and allied discipline through continuous learning
- M2. To foster moral values in students and equip them for developing sustainable solutions to serve both national and global needs in society and industry.
- M3. To digitize educational resources and process for enhanced teaching and effective learning.
- M4. To cultivate an environment supporting incubation, product development, technology transfer, capacity building and entrepreneurship.
- M5. To encourage faculty-student networking with alumni, industry, institutions, and other stakeholders for collective engagement.



Department of Computer Science & Engineering

VISION

"To attain global excellence through education, innovation, research, and work ethics in the field of Computer Science and engineering with the commitment to serve humanity."

MISSION

M1 To lead in the advancement of computer science and engineering through internationally recognized research and education.

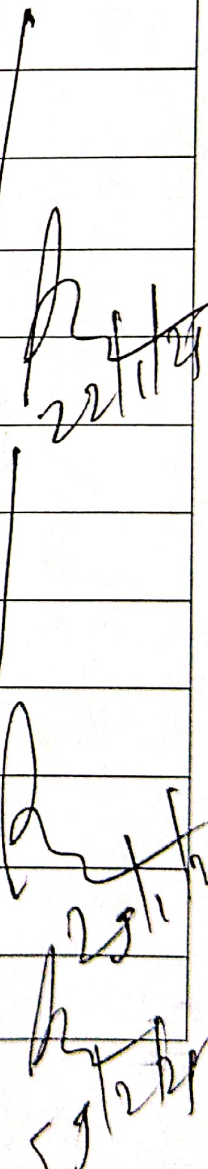
M2 To prepare students for full and ethical participation in a diverse society and encourage lifelong learning.

M3 To foster development of problem solving and communication skills as an integral component of the profession.

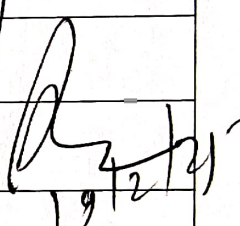
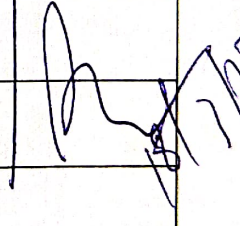
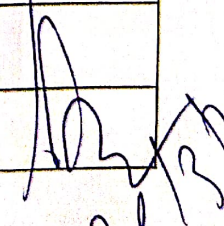
M4 To impart knowledge, skills and cultivate an environment supporting incubation, product development, technology transfer, capacity building and entrepreneurship in the field of computer science and engineering.

M5 To encourage faculty, student's networking with alumni, industry, institutions, and other stakeholders for collective engagement

LAB INDEX

S.No	Experiment	R1	R2	R3	R4	R5	Total Marks (10)	DATE	Faculty Signature
		Is able to identify and define the objective of the given problem?	Is proposed design/procedure/algorithm solves the problem?	Has the understanding of the tool/programming language to implement the proposed solution?	Are the result(s) verified using sufficient test data to support the conclusions?	Individuality of submission?			
		2 Marks	2 Marks	2 Marks	2 Marks	2 Marks			
1	WAP To Implement Multiple Inheritance							22-01-25	
2	WAP To Implement Labelled Loops							22-01-25	
3	WAP To Implement Interfaces							22-01-25	
4	WAP To Implement Multi-level Inh.							22-01-25	
5	WAP To Implement dynamic method disp.							22-01-25	
6	WAP for Command Line Arguments							29-01-25	
7	WAP To Implement Class In Java							29-01-25	
8	WAP To Implement Single Inheritance							29-01-25	
9	WAP To Implement Constructors							29-01-25	
10	WAP To Implement Method Overloading							29-01-25	
11	Special Question for Practice							29-01-25	
12	Loop on The Threads In Java							19-02-25	

LAB INDEX

S.No	Experiment	R1	R2	R3	R4	R5	Total Marks (10)	Date	Faculty Signature
		Is able to identify and define the objective of the given problem?	Is proposed design /procedure /algorithm solves the problem?	Has the understanding of the tool/programming language to implement the proposed solution?	Are the result(s) verified using sufficient test data to support the conclusions?	Individuality of submission?			
		2 Marks	2 Marks	2 Marks	2 Marks	2 Marks			
13.	WAP for Synchronize of Threads							19-02-25	
14.	Producer-Consumer Problem In Thread							19-02-25	
15.	Exception Handling In Java							19-02-25	
16.	WAP To Implement Applet In Java							19-03-25	
17.	WAP To Implement Moving Barrel							19-03-25	
18.	WAP To Call on frame							19-03-25	
19.	WAP To Implement Student Java Bean							19-03-25	
20.	WAP To Implement Employee Java Bean							19-03-25	
21.	WAP To Implement Web Servlet							26-03-25	
22.	WAP To Implement Generic Servlet							26-03-25	
23.	WAP To Show Lifecycle of Servlet							26-03-25	
24.	WAP For dynamic Servlet Routes							26-03-25	

26/3/25

LAB INDEX

[illegible]

LAB INDEX

[illegible]

Lab Assessment Sheet

[illegible]

Lab Assessment Sheet

[illegible]

Lab Exercise-1. (22/01/2025)

Q :- WAP To Implement multiple inheritance?

Multiple inheritance in Java refers to a class inheriting features from more than one parent class. Java **does not support multiple inheritance with classes** to avoid ambiguity caused by the "Diamond Problem." However, Java supports multiple inheritance using **interfaces**, allowing a class to implement multiple interfaces. This ensures flexibility while maintaining simplicity and avoiding conflicts in method resolution.

CODE :-

```
interface Flyable {  
    void fly();  
}  
  
interface Swimmable {  
    void swim();  
}  
  
class Duck implements Flyable, Swimmable {  
    @Override  
    public void fly() {  
        System.out.println("Duck flies in a specific way!");  
    }  
  
    @Override  
    public void swim() {  
        System.out.println("Duck swims in a specific way!");  
    }  
}
```

```
public class AdvJava_001_MultipleInheritance {  
    public static void main(String[] args) {  
        Duck duck = new Duck();  
        duck.fly();  
        duck.swim();  
    }  
}
```

OUTPUT : -

```
PS E:\Yash Pandey\04_YashPandey_Codes(College)\008_Advanced-Java-Codes> cd "e:\Yash Pandey\04_YashPandey_Codes(College)\008_Advanced-Java-Codes" & java AdvJava_001_MultipleInheritance.java } ; if ($?) { java AdvJava_001_MultipleInheritance }
```

```
Duck flies in a specific way!  
Duck swims in a specific way!
```

Lab Exercise-2. (22/01/2025)

Q :- WAP To show the use of labeled loops?

Labeled loops in Java allow you to assign a label to a loop, enabling control flow statements like break or continue to directly target and control the labeled loop. This is particularly useful in nested loops, where you can break or continue an outer loop from within an inner loop, avoiding the need for complex logic. The label helps to identify which loop to affect, making the code more **readable and efficient** when dealing with multiple nested loops.

CODE :-

```
public class AdvJava_002_LabelledLoops {  
    public static void main(String[] args) {  
        outerLoop: // Label for outer loop  
        for (int i = 1; i <= 3; i++) {  
            innerLoop: // Label for inner loop  
            for (int j = 1; j <= 3; j++) {  
                if (i == 2 && j == 2) {    System.out.println("Skipping i = " + i + ", j = " + j);    }  
                System.out.println("i = " + i + ", j = " + j);  
            }  
        }  
    }  
}
```

OUTPUT :-

```
PS E:\Yash Pandey\04_YashPandey_Codes(College)\008_Advanced-Java-Codes\0  
001_Classes-In-Java\" ; if ($?) { javac AdvJava_002_LabelledLoops.java }  
i = 1, j = 1  
i = 1, j = 2  
i = 1, j = 3  
i = 2, j = 1  
Skipping i = 2, j = 2  
i = 3, j = 1  
i = 3, j = 2  
i = 3, j = 3
```

Lab Exercise-3. *(22/01/2025)*

Q :- WAP to show all types of Inheritance among Interfaces?

In Java, Interfaces in Java are fully abstract, meaning they can only contain method declarations (default, static, and private methods can have implementations). They provide a blueprint for classes, ensuring a contract for implementing classes to follow. Since Java does not allow multiple inheritance with classes, interfaces enable multiple inheritance by allowing a class to implement multiple interfaces, promoting flexibility and code reusability.

Inheritance among interfaces allows for the reuse and extension of functionality without code duplication. Interfaces support **single inheritance**, where one interface extends another, inheriting its methods. They also support **multiple inheritance**, enabling an interface to inherit from multiple interfaces, combining their behaviours. Additionally, **multilevel inheritance** allows an interface to extend another interface that itself extends one or more interfaces, forming a chain of inheritance. This flexibility makes interfaces a powerful tool for designing modular and scalable applications.

1. Single Inheritance:

- One interface inherits another interface.
- Example: **interface B extends A.**

2. Multiple Inheritance:

- An interface inherits from two interfaces & multiple sources.
- Example: **interface C extends A, D.**

3. Multilevel Inheritance:

- An interface inherits from another interface, which itself extends one or more interfaces & Forms a chain of inheritance.
- Example: **interface E extends C** (where C extends A and D).

CODE :-

// Single inheritance: Interface A is inherited by Interface B

```
interface A {  
    void methodA();  
}
```

```
interface B extends A {  
    void methodB();  
}
```

// Multiple inheritance: C inherits from A and D

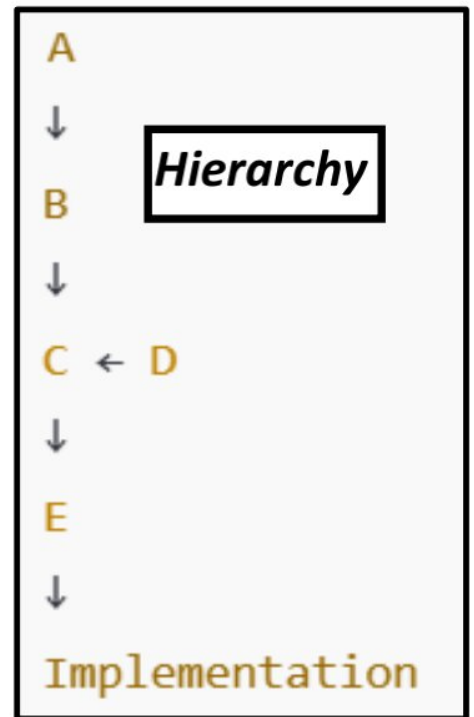
```
interface D {  
    void methodD();  
}  
  
interface C extends A, D {  
    void methodC();  
}
```

// Multilevel inheritance: Interface E inherits from Interface C

```
interface E extends C {  
    void methodE();  
}
```

// Implementing all the interfaces in a single class

```
class Implementation implements E {  
  
    @Override  
    public void methodA() {  
        System.out.println("Method A from Interface A");  
    }  
  
    @Override  
    public void methodB() {  
        System.out.println("Method B from Interface B");  
    }  
}
```



```

    }

    @Override
    public void methodC() {
        System.out.println("Method C from Interface C");
    }

    @Override
    public void methodD() {
        System.out.println("Method D from Interface D");
    }

    @Override
    public void methodE() {
        System.out.println("Method E from Interface E");
    }
}

public class Main {
    public static void main(String[] args) {
        Implementation obj = new Implementation();

        obj.methodA();    obj.methodB();

        obj.methodC();    obj.methodD();    obj.methodE();
    }
}

```

OUTPUT :-

```

PS E:\Yash Pandey\04_YashPandey_Codes(College)\008_Advanced-Java-Codes/
003_Interface-In-Java\"; if ($?) { javac AdvJava_003_Interface-In-Java

```

```

Method A from Interface A
Method B from Interface B
Method C from Interface C
Method D from Interface D
Method E from Interface E

```

Lab Exercise-4. (22/01/2025)

Q :- WAP To show Multilevel Inheritance?

Multilevel inheritance in Java occurs when a class inherits from a class that is already derived from another class. This creates a chain of inheritance where properties and methods are passed down through multiple levels. It allows a subclass to inherit features from both its immediate parent and the ancestor classes, enabling reusability and extensibility of code. However, care should be taken to avoid excessive levels, which can make the hierarchy complex and harder to manage.

CODE : -

```
class GrandFather{
    String name;          int age;
    public GrandFather(String name, int age) {
        this.name = name;          this.age=age;
        System.out.println("Executing the constructor of the GrandFather class.");
    }
    public void Display(){
        System.out.println("GrandFather's name is '"+name+"' and age is "+age);
    }
}

class Father extends GrandFather{
    String name;          int age;
    public Father(String Gname,int Gage,String name, int age) {
        super(Gname, Gage);    this.name = name;    this.age=age;
        System.out.println("Executing the constructor of the Father class.");
    }
    public void Display(){
        super.Display();
    }
}
```

```

        System.out.println("Father's name is '"+name+"' and age is "+age);
    }
}

class Son extends Father{
    String name;  int age;
    public Son(String Gname,int Gage,String Pname,int Page,String name, int age) {
        super(Gname, Gage, Pname, Page);
        this.name = name;    this.age=age;
        System.out.println("Executing the constructor of the Son class.");
    }
    public void Display(){
        super.Display();
        System.out.println("Son's name is '"+name+"' and age is "+age);
    }
}

public class AdvJava_003_MultiLevelInheritance {
    public static void main(String[] args) {
        Son son = new Son("Hira Lal",84,"Prabhash",50,"Yash",21);
        son.Display();
    }
}

```

OUTPUT :-

```

PS E:\Yash Pandey\04_YashPandey_Codes(College)\008_Advanced-Java-Codes\001_Classes-
001_Classes-In-Java\" ; if ($?) { javac AdvJava_003_MultiLevelInheritance.java } ;
Executing the constructor of the GrandFather class.
Executing the constructor of the Father class.
Executing the constructor of the Son class.

GrandFather's name is 'Hira Lal' and age is 84
Father's name is 'Prabhash' and age is 50
Son's name is 'Yash' and age is 21

```

Lab Exercise-5. (22/01/2025)

Q :- WAP To show dynamic method dispatch?

Dynamic method dispatch in Java is a mechanism where the method to be executed is determined at runtime, based on the object being referred to. It is achieved through method overriding and polymorphism. When a superclass reference points to a subclass object, the overridden method in the subclass is called, not the one in the superclass. This allows Java to support runtime polymorphism and enables flexible and extensible code behavior.

CODE :-

```
class Animal {  
    void sound() {  
        System.out.println("Animal makes a sound");  
    }  
}
```

```
class Dog extends Animal {  
    @Override  
    void sound() {  
        System.out.println("Dog barks");  
    }  
}
```

```
class Cat extends Animal {  
    @Override  
    void sound() {  
        System.out.println("Cat meows");  
    }  
}
```

```
public class AdvJava_004_DynamicMethod {  
    public static void main(String[] args) {  
        Animal animal = new Dog();  
        animal.sound();  
  
        animal = new Cat();  
        animal.sound();  
    }  
}
```

OUTPUT : -

```
PS E:\Yash Pandey\04_YashPandey_Codes(College)\008_Advanced-Java-Codes\0  
001_Classes-In-Java\" ; if ($?) { javac AdvJava_004_DynamicMethod.java }  
Dog barks  
Cat meows
```

Lab Exercise-6. (29/01/2025)

Q :- WAP To show command-line arguments?

In Java, command-line arguments are passed to the main method as an array of strings: `public static void main(String[] args)`. These arguments allow users to provide input values when executing the program from the terminal. Each argument is stored as a string in the `args` array and can be accessed using indexing (e.g., `args[0]` for the first argument). They are commonly used for dynamic input handling, configuration settings, and automation.

CODE :-

```
public class AdvJava_001_Command_Line_Arguments {  
    public static void main(String[] args) {  
        if (args.length > 0) {  
            System.out.println("Command Line Arguments: ");  
            for (int i = 0; i < args.length; i++) {  
                System.out.println("Argument " + (i + 1) + ": " + args[i]);  
            }  
        } else {  
            System.out.println("No command line arguments passed.");  
        }  
    }  
}
```

OUTPUT :-

```
● PS E:\Yash Pandey\04_YashPandey_Codes(College)\008_Advanced-Java-Codes> javac temp.java  
● PS E:\Yash Pandey\04_YashPandey_Codes(College)\008_Advanced-Java-Codes> java temp YashPandey 29 122  
Command Line Arguments:  
Argument 1: YashPandey  
Argument 2: 29  
Argument 3: 122
```

Lab Exercise-7. *(29/01/2025)*

Q :- WAP To create a class with variable and method , Then call those methods and variables by using the objects Of That Class?

In Java, a **class** is a blueprint for creating objects, defining their attributes (fields) and behaviors (methods). An **object** is an instance of a class, created using the new keyword. Each object has its own copy of instance variables and can access methods defined in the class. Classes **promote code reusability** and encapsulation, making Java a powerful object-oriented programming language.

CODE : -

```
class Student{  
  
    private String name;  
  
    private int age;  
  
    private int Marks;  
  
    Student(String name,int age,int Marks){  
  
        this.name=name;  
  
        this.age=age;  
  
        this.Marks=Marks;  
  
    }  
  
    String ReturnName(){  
  
        return name;  
  
    }  
  
    int ReturnAge(){  
  
        return age;  
  
    }  
}
```

```
void Display(){
    System.out.println("Hi "+name+" , You're "+age+" Years Old &
                        Your Overall Marks Are "+Marks);
}
}

public class AdvJava_002_Classes_Objects{
    public static void main(String[] args) {
        Student Yash=new Student("Yash", 21, 95);
        System.out.println("Your Name Is ::: "+Yash.ReturnName());
        System.out.println("Your Age Is ::: "+Yash.ReturnAge());
        Yash.Display();
    }
}
```

OUTPUT :-

```
● PS E:\Yash Pandey\04_YashPandey_Codes(College)\008_Advanced-Java-Codes>
asses_Objects.java } ; if ($?) { java AdvJava_002_Classes_Objects }
Your Name Is ::: Yash
Your Age Is ::: 21
Hi Yash , You're 21 Years Old & Your Overall Marks Are 95
```

Lab Exercise-8. *(29/01/2025)*

Q :- WAP To demonstrate single inheritance?

Single inheritance in Java is a mechanism where a subclass inherits the properties and behaviors of a single superclass. This **allows code reusability** and **promotes hierarchical classification**. The subclass gains access to the parent class's fields and methods, which can be used directly or overridden to provide specific implementations. It is implemented using the `extends` keyword. Single inheritance helps in maintaining a clear and simple class structure while avoiding complexity.

CODE :-

```
class Father{
    String name;
    int age;
    public Father(String name, int age) {
        this.name = name;
        this.age=age;
        System.out.println("Executing the constructor of the Father class.");
    }
    public void Display(){
        System.out.println("Father's name is '"+name+"' and age is "+age);
    }
}

class Son extends Father{
    String name;
    int age;
    public Son(String Pname,int Page,String name, int age) {
        super(Pname, Page);
```

```

        this.name = name;

        this.age=age;

        System.out.println("Executing the constructor of the Son class.");
    }

    public void Display(){

        super.Display();

        System.out.println("Son's name is '"+name+"' and age is "+age);
    }
}

public class AdvJava_003_Single_Inheritance{

    public static void main(String[] args) {

        Son son = new Son("Prabhash",50,"Yash",21);

        son.Display();

    }
}

```

OUTPUT :-

```

● PS E:\Yash Pandey\04_YashPandey_Codes(College)\008_Advanced-Java-Codes>
nple_Inheritance.java } ; if ($?) { java AdvJava_003_Single_Inheritance
Executing the constructor of the Father class.
Executing the constructor of the Son class.

Father's name is 'Prabhash' and age is 50
Son's name is 'Yash' and age is 21

```

Lab Exercise-9. (29/01/2025)

Q :- WAP To show the Constructors in Java ?

In Java, a **constructor** is a special method used to initialize objects when they are created. It has the same name as the class and does not have a return type. Constructors are called automatically when an object is instantiated using the new keyword. Java provides **default constructors** (if none is defined) and allows **parameterized constructors** for custom initialization. They help in setting initial values for object attributes.

CODE : -

```
class Car {  
    String model;  
    int year;  
  
    // Default constructor  
    Car() {  
        System.out.println("Excuting The Default Constructor...");  
        model = "Unknown";  
        year = 0;  
    }  
  
    // Parameterized constructor  
    Car(String model, int year) {  
        System.out.println("Excuting The Parameterized Constructor...");  
        this.model = model;  
        this.year = year;  
    }  
  
    // Copy constructor  
    Car(Car other) {  
        System.out.println("Excuting The Copy Constructor...");  
        this.model = other.model;
```

```

        this.year = other.year;
    }

    void display() {
        System.out.println("Model: " + model);
        System.out.println("Year: " + year);
    }
}

public class AdvJava_004_Constructors {
    public static void main(String[] args) {
        Car car1 = new Car();
        car1.display();

        Car car2 = new Car("Tesla Model S", 2023);
        car2.display();

        Car car3 = new Car(car2);
        car3.display();
    }
}

```

OUTPUT :-

```

● PS E:\Yash Pandey\04_YashPandey_Codes(College)\008_Advanced-Java-Codes>
nstructors.java } ; if ($?) { java AdvJava_004_Constructors }
Exicuting The Default Constructor....
Model: Unknown
Year: 0
Exicuting The Parameterized Constructor....
Model: Tesla Model S
Year: 2023
Exicuting The Copy Constructor....
Model: Tesla Model S
Year: 2023

```

Lab Exercise-10. (29/01/2025)

Q :- WAP To show method overloading ?

Method overloading in Java is a feature that allows multiple methods in the same class to have the same name but different parameter lists (different number, type, or order of parameters). It enables code reusability and improves readability by allowing to perform similar tasks with different inputs.

CODE :-

```
public class AdvJava_005_Method_Overloading {  
    public static void Area(int l,int b){  
        System.out.println("The Area Of Rectangle Is ::: "+(l*b));  
    }  
    public static void Area(int side){  
        System.out.println("The Area Of Square Is ::: "+(side*side));  
    }  
    public static void Area(double radius){  
        System.out.println("The Area Of Circle Is ::: "+(3.14*radius*radius));  
    }  
    public static void main(String[] args) {  
        Area(10,20);    Area(10);    Area(5.5);  
    }  
}
```

OUTPUT :-

```
● PS E:\Yash Pandey\04_YashPandey_Codes(College)\008_Advanced-Java-Codes>  
thod_Overloading.java } ; if ($?) { java AdvJava_005_Method_Overloading  
The Area Of Rectangle Is ::: 200  
The Area Of Square Is ::: 100  
The Area Of Circle Is ::: 94.985
```

Lab Exercise-11. *(29/01/2025)*

Q :- WAP To Show same parameter list and different parameter list in parent-child class.
Also, specify the category of this program?

The program falls into the category of **Method Overloading** and **Method Overriding** in Java. Method overloading occurs when multiple methods with the same name but different parameter are defined in the same sub-class .

Method overriding happens when a subclass provides its own implementation of a method that is already defined in the parent class, with the same method signature (name and parameter list).

This **program demonstrates both concepts** by showcasing how methods can be redefined or overloaded in a parent-child class hierarchy. Method overloading allows the same method name with different parameter lists, while method overriding enables the child class to provide its own implementation of a parent method.

CODE : -

```
class Parent {  
    void show(int a) {  
        System.out.println("Parent class method with one parameter: " + a);  
    }  
    void show(int a, int b) {  
        System.out.println("Parent class method with two parameters: " + a + ", " + b);  
    }  
}  
  
class Child extends Parent {  
    @Override
```

```

void show(int a) {
    System.out.println("Child class method (Overriding) with one parameter: " + a);
}

void show(String msg) {
    System.out.println("Child class method (Overloading) with String parameter: " + msg);
}
}

public class AdvJava_006_SpecialQuestion {
    public static void main(String[] args) {
        Child obj = new Child();
        obj.show(10);
        obj.show("Hello");
        obj.show(10, 20);
    }
}

```

OUTPUT :-

```

PS E:\Yash Pandey\04_YashPandey_Codes(College)\008_Advanced-Java-Codes>
ecialQuestion.java } ; if ($?) { java AdvJava_006_SpecialQuestion }
Child class method (Overriding) with one parameter: 10
Child class method (Overloading) with String parameter: Hello
Parent class method with two parameters: 10, 20

```

Lab Exercise-12. *(19/02/2025)*

Q :- WAP To Implement for Loop & Methods Of Thread like join,wait,sleep and suspend?

In Java, threads are the smallest unit of execution within a program, allowing for multitasking and parallelism. A thread represents an independent path of execution that runs concurrently with other threads in the same program. Java provides the Thread class and the Runnable interface to manage threads.

Developers can create threads by either extending the Thread class or implementing the Runnable interface and passing it to a Thread object. Threads in Java can be started using the start() method, and their execution can be controlled using methods like sleep(), wait(), and notify(). Java's multithreading capabilities are especially useful for optimizing CPU usage, improving performance in tasks like file I/O or handling multiple requests in server applications.

CODE : -

```
class MyThread extends Thread {  
    public void run() {  
        try {  
            System.out.println(Thread.currentThread().getName() + " started.");  
            Thread.sleep(1000);  
            System.out.println(Thread.currentThread().getName() + " slept for 1 second.");  
  
            synchronized (this) {  
                wait(1000);  
                System.out.println(Thread.currentThread().getName() + " resumed after waiting.");  
            }  
        } catch (InterruptedException e) {  
            System.out.println(e);  
        }  
    }  
}
```

```

    }
}

@SuppressWarnings("removal")
public void suspendThread() {
    try {
        Thread.sleep(500);
        System.out.println(Thread.currentThread().getName() + " is going to suspend.");
        this.suspend();
        System.out.println(Thread.currentThread().getName() + " is suspended.");
    } catch (InterruptedException e) {
        System.out.println(e);
    }
}

}

public class AdvJava001_BasicThreadMethods {
    public static void main(String[] args) {
        MyThread t1 = new MyThread();
        MyThread t2 = new MyThread();

        for (int i = 0; i < 2; i++) {
            if (i == 0) {
                t1.start();
            } else {
                t2.start();
            }
        }
    }
}

```

```
try {  
    t1.join();  
    t2.join();  
    System.out.println("Both threads have finished.");  
} catch (InterruptedException e) {  
    System.out.println(e);  
}  
  
t1.suspendThread();  
}  
}
```

OUTPUT : -

```
PS C:\Users\mait\Desktop\YashPandey122> cd "c:\Users\mait\Desktop\YashPandey122\" ;  
Thread-0 started.  
Thread-1 started.  
Thread-1 slept for 1 second.  
Thread-0 slept for 1 second.  
Thread-0 resumed after waiting.  
Thread-1 resumed after waiting.  
Both threads have finished.  
main is going to suspend.
```

Lab Exercise-13. *(19/02/2025)*

Q :- WAP To Implement Synchronization In Threads And Remove Deadlock?

In Java, threads are the smallest unit of execution within a program, allowing for multitasking and parallelism. A thread represents an independent path of execution that runs concurrently with other threads in the same program. Java provides the Thread class and the Runnable interface to manage threads.

Synchronization in threads in Java is a mechanism that ensures that only one thread can access a shared resource at a time, preventing data inconsistencies or corruption. It is achieved using the synchronized keyword, which can be applied to methods or blocks of code. When a thread enters a synchronized method, it locks the corresponding object or class and other threads must wait until the lock is released. This is crucial in multi-threaded environments where multiple threads access shared resources, like variables or data structures, concurrently.

CODE : -

```
class Resource {  
    public synchronized void methodA(Resource other) {  
        System.out.println(Thread.currentThread().getName() + " is in methodA.");  
        try {  
            Thread.sleep(1000);  
            other.methodB(this);  
        } catch (InterruptedException e) {  
            System.out.println(e);  
        }  
    }  
}
```

```

public synchronized void methodB(Resource other) {
    System.out.println(Thread.currentThread().getName() + " is in methodB.");
    try {
        Thread.sleep(1000);
        other.methodA(this);
    } catch (InterruptedException e) {
        System.out.println(e);
    }
}
}

```

```

public class AdvJava002_DeadlockPrevention {
    public static void main(String[] args) {
        Resource resource1 = new Resource();
        Resource resource2 = new Resource();

        Thread t1 = new Thread(() -> {
            synchronized (resource1) {
                System.out.println(Thread.currentThread().getName() + " locked resource1.");
                try { Thread.sleep(100); } catch (InterruptedException e) {}
            }
            synchronized (resource2) {
                System.out.println(Thread.currentThread().getName() + " locked resource2.");
                resource1.methodA(resource2);
            }
        });

        Thread t2 = new Thread(() -> {
            synchronized (resource1) {

```

```

        System.out.println(Thread.currentThread().getName() + " locked resource1.");
    try { Thread.sleep(100); } catch (InterruptedException e) {}
    synchronized (resource2) {
        System.out.println(Thread.currentThread().getName() + " locked resource2.");
        resource1.methodB(resource2);
    }
}
});

t1.start();
t2.start();
}
}

```

OUTPUT :-

```

PS C:\Users\mait\Desktop\YashPandey122> cd "c:\Users\mait\Desktop\YashPandey122\" ;
Thread-0 locked resource1.
Thread-0 locked resource2.
Thread-0 is in methodA.
Thread-0 is in methodB.
Thread-0 is in methodA.
Thread-0 is in methodB.
Thread-0 is in methodA.
Thread-0 is in methodB.
Thread-0 is in methodA.
Thread-0 is in methodB.
Thread-0 is in methodA.
Thread-0 is in methodB.
Thread-0 is in methodA.
Thread-0 is in methodB.
Thread-0 is in methodA.
Thread-0 is in methodB.
Thread-0 is in methodA.
Thread-0 is in methodB.
PS C:\Users\mait\Desktop\YashPandey122> █

```

Lab Exercise-14. (19/02/2025)

Q :- WAP To Implement Producer Consumer Problem In Java?

In Java, threads are the smallest unit of execution within a program, allowing for multitasking and parallelism. A thread represents an independent path of execution that runs concurrently with other threads in the same program. Java provides the Thread class and the Runnable interface to manage threads.

The Producer-Consumer problem is a classic synchronization problem in concurrent programming where two threads, the producer and the consumer, share a common resource (such as a buffer). The producer generates data and stores it in the buffer, while the consumer takes data from the buffer. To avoid race conditions and ensure thread safety, synchronization mechanisms like wait() and notify() are used to coordinate the access to the shared resource, ensuring that the producer doesn't produce more data than the consumer can consume, and vice versa. This problem often illustrates the need for proper coordination and managing shared resources between threads.

CODE :-

```
import java.util.LinkedList;

import java.util.Queue;

class SharedBuffer {

    private final Queue<Integer> buffer = new LinkedList<>();

    private final int MAX_CAPACITY = 5;

    public synchronized void produce() throws InterruptedException {

        while (buffer.size() == MAX_CAPACITY) {

            wait();
```

```
    }  
    int item = (int) (Math.random() * 100);  
    buffer.add(item);  
    System.out.println("Produced: " + item);  
    notify();  
}
```

```
public synchronized void consume() throws InterruptedException {  
    while (buffer.isEmpty()) {  
        wait();  
    }  
    int item = buffer.poll();  
    System.out.println("Consumed: " + item);  
    notify();  
}  
}
```

```
class Producer implements Runnable {  
    private final SharedBuffer sharedBuffer;  
  
    public Producer(SharedBuffer sharedBuffer) {  
        this.sharedBuffer = sharedBuffer;  
    }  
  
    @Override  
    public void run() {  
        try {  
            while (true) {  
                sharedBuffer.produce();  
            }  
        }  
    }  
}
```

```
        Thread.sleep(1000);
    }
} catch (InterruptedException e) {
    System.out.println(e);
}
}
}
```

```
class Consumer implements Runnable {
    private final SharedBuffer sharedBuffer;

    public Consumer(SharedBuffer sharedBuffer) {
        this.sharedBuffer = sharedBuffer;
    }
}
```

```
@Override
public void run() {
    try {
        while (true) {
            sharedBuffer.consume();
            Thread.sleep(1500);
        }
    } catch (InterruptedException e) {
        System.out.println(e);
    }
}
}
```

```
public class AdvJava003_ProducerConsumerProblem {  
    public static void main(String[] args) {  
        SharedBuffer sharedBuffer = new SharedBuffer();  
  
        Thread producerThread = new Thread(new Producer(sharedBuffer));  
        Thread consumerThread = new Thread(new Consumer(sharedBuffer));  
  
        producerThread.start();  
        consumerThread.start();  
    }  
}
```

OUTPUT :-

```
PS C:\Users\mait\Desktop\YashPandey122> cd "c:\Users\mait\Desktop\YashPandey122\" ;  
Produced: 65  
Consumed: 65  
Produced: 14  
Consumed: 14  
Produced: 22  
Produced: 21  
Consumed: 22  
Produced: 42  
Consumed: 21  
Produced: 51  
Consumed: 42  
Produced: 76  
Produced: 35  
Consumed: 51  
Produced: 28  
PS C:\Users\mait\Desktop\YashPandey122> █
```

Lab Exercise-15. (19/02/2025)

Q :- WAP To Implement Try-Catch-Finally Block In Java?

The try-catch-finally block in Java is used to handle exceptions and ensure that necessary cleanup actions are performed, regardless of whether an exception occurs. The try block contains the code that might throw an exception, while the catch block is used to handle any exceptions that occur within the try block. If no exceptions are thrown, the catch block is skipped. The finally block is optional but is always executed after the try and catch blocks, whether an exception was thrown or not, making it ideal for cleanup operations such as closing files or releasing resources.

CODE :-

```
public class AdvJava004_TryCatch {  
    public static void main(String[] args) {  
        int arr[] = { 1, 2, 3, 4, 5 };  
        int n = arr.length;  
  
        try {  
            System.out.println("Currently In Try Block");  
            checkArrayIndex(arr, n);  
        } catch (Exception e) {  
            System.out.println("\nCurrently In Catch Block");  
            System.out.println(e.getMessage());  
        } finally {  
            System.out.println("This is finally block, Program Has Ended");  
        }  
    }  
}
```

```
public static void checkArrayIndex(int[] arr, int n) throws Exception {  
    for (int i = 0; i <= n; i++) {  
        if (i >= arr.length) {  
            throw new ArrayIndexOutOfBoundsException("Array index out of bounds: " + i);  
        }  
        System.out.print(arr[i] + " ");  
    }  
}  
}
```

OUTPUT :-

```
PS C:\Users\mait\Desktop\YashPandey122> cd "c:\Users\mait\Desktop\YashPandey122\" ;  
Currently In Try Block  
1 2 3 4 5  
  
Currently In Catch Block  
Array index out of bounds: 5  
  
This is finally block, Program Has Ended  
PS C:\Users\mait\Desktop\YashPandey122> █
```

Lab Exercise-16. *(19/03/2025)*

Q :- WAP To Implement An Basic Applet Program In Java?

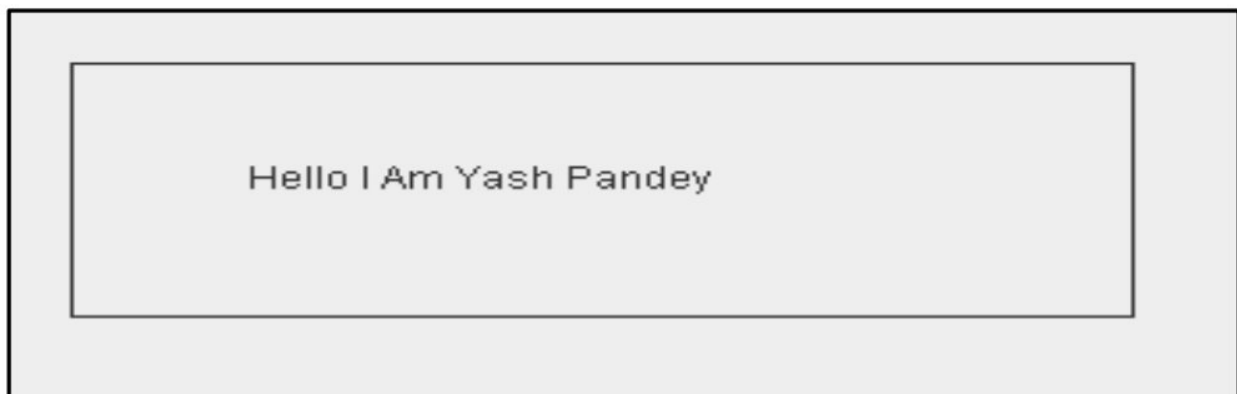
Applet: An applet in Java is a small application designed to be embedded within a web page and executed in a web browser. It extends the Applet class and allows developers to create interactive and graphical user interfaces (GUIs) that can run within a browser's Java Virtual Machine (JVM). Applets can respond to user inputs, handle events, and update their visuals dynamically. However, due to security concerns and the rise of modern web technologies, applets are deprecated and no longer supported in most browsers.

CODE : -

```
import java.applet.*;
import java.awt.*;

public class AdvJava001_BasicApplet extends Applet {
    @Override
    public void paint(Graphics g) {
        g.drawRect(50, 50, 300, 100);
        g.drawString("Hello I Am Yash Pandey", 100, 100);
    }
}
```

OUTPUT : -



Lab Exercise-17. *(19/03/2025)*

Q :- WAP To Implement An Moving Banner Program In Java Using Applet And AWT ?

Applet: An applet in Java is a small application designed to be embedded within a web page and executed in a web browser. It extends the Applet class and allows developers to create interactive and graphical user interfaces (GUIs) that can run within a browser's Java Virtual Machine (JVM). Applets can respond to user inputs, handle events, and update their visuals dynamically. However, due to security concerns and the rise of modern web technologies, applets are deprecated and no longer supported in most browsers.

AWT (Abstract Window Toolkit): AWT is a Java library that provides a set of graphical user interface (GUI) components and tools for building cross-platform applications. It includes classes for creating windows, buttons, menus, and other graphical elements. AWT components are platform-dependent because they rely on the native system's windowing system to render the UI, making it less flexible than newer libraries like Swing and JavaFX, which provide a more consistent appearance across platforms.

CODE : -

```
import java.applet.*;
```

```
import java.awt.*;
```

```
public class AdvJava002_MovingBanner extends Applet implements Runnable {  
    private String message = "Welcome to the Moving Banner!";  
    private int xPos = 300;  
    private int yPos = 100;  
    private Thread thread;
```

```

@Override
public void init() {
    thread = new Thread(this);        thread.start();
}

@Override
public void run() {
    while (true) {
        xPos -= 2;
        if (xPos < -getFontMetrics(getFont()).stringWidth(message)) {
            xPos = getWidth();
        }        repaint();
        try {            Thread.sleep(50);        }
        catch (InterruptedException e) {        Thread.currentThread().interrupt();        }
    }
}

@Override
public void paint(Graphics g) {
    g.clearRect(0, 0, getWidth(), getHeight());
    g.drawString(message, xPos, yPos);
}
}

```

OUTPUT :-



Welcome to the Moving Banner!

Lab Exercise-18. *(19/03/2025)*

Q :- WAP To Implement An Program To Call An Frame Using Applet And AWT ?

Applet: An applet in Java is a small application designed to be embedded within a web page and executed in a web browser. It extends the Applet class and allows developers to create interactive and graphical user interfaces (GUIs) that can run within a browser's Java Virtual Machine (JVM). Applets can respond to user inputs, handle events, and update their visuals dynamically. However, due to security concerns and the rise of modern web technologies, applets are deprecated and no longer supported in most browsers.

AWT (Abstract Window Toolkit): AWT is a Java library that provides a set of graphical user interface (GUI) components and tools for building cross-platform applications. It includes classes for creating windows, buttons, menus, and other graphical elements. AWT components are platform-dependent because they rely on the native system's windowing system to render the UI, making it less flexible than newer libraries like Swing and JavaFX, which provide a more consistent appearance across platforms.

Frame: In Java, a **Frame** is a top-level container used to create a window for GUI applications. It is part of the AWT (Abstract Window Toolkit) and is used to display graphical components like buttons, labels, and text fields. A Frame can be customized by setting its size, title, and visibility. While Frame provides a basic window for applications, it is often replaced by more .

An Frame object is instantiated and customized with a title, size, and visibility. A Label is added to the frame, and the frame is displayed as a separate window outside the applet's default rendering area. While applets were designed to be embedded within a webpage, this approach demonstrates how to use a frame to create an independent window for GUI components. However, with modern Java practices, using JFrame in Swing for standalone applications is preferred due to better flexibility and support across platforms.

CODE :-

```
import java.applet.*;
import java.awt.*;

public class AdvJava003_AppletWithFrame extends Applet {
    @Override
    public void init() {
        Frame myFrame = new Frame("Applet with Frame");
        Label label = new Label("Hello, I am inside a Frame!");
        myFrame.add(label);
        myFrame.setSize(300, 200);
        myFrame.setVisible(true);
    }
    @Override
    public void paint(Graphics g) {
        g.drawString("This is an Applet!", 50, 50);
    }
}
```

OUTPUT :-



Hello, I am inside a Frame!

Lab Exercise-19. *(19/03/2025)*

Q :- WAP To Implement Student JavaBean And Access It Using Another Java File ?

A **JavaBean** is a reusable, encapsulated Java class that follows specific conventions, making it easy to use in various applications. It must have a no-argument constructor, be serializable, and provide getter and setter methods for accessing private fields. JavaBeans are widely used in enterprise applications, particularly in Java EE, for managing data and enabling smooth communication between different application layers. They enhance modularity and maintainability, making them ideal for frameworks like Spring and JSP.

CODE : -

1. StudentBean.java File

```
package student_javabean;

import java.io.Serializable;

public class Student implements Serializable {

    private static final long serialVersionUID = 1L;

    private int id;        private int age;

    private String name;

    public Student() {}

    public Student(int id, String name, int age) {

        this.id = id;

        this.name = name;

        this.age = age;

    }

    public int getId() {    return id;    }

    public void setId(int id) {    this.id = id;    }
```

```

public String getName() {    return name;    }

public void setName(String name) {    this.name = name;    }

public int getAge() {    return age;    }

public void setAge(int age) {
    if (age > 0) {        this.age = age;    }
    else {        System.out.println("Age must be positive.");    }
}

@Override

public String toString() {
    return "Student [ID=" + id + ", Name=" + name + ", Age=" + age + "];"
}
}

```

2. StudentTest.java File

```

package student_javabean;

public class StudentTest {

    public static void main(String[] args) {

        // Creating a Student object using no-arg constructor
        Student student = new Student();

        student.setId(101);
        student.setName("Alice");
        student.setAge(20);

        // Displaying student details
        System.out.println("Student ID: " + student.getId());
        System.out.println("Student Name: " + student.getName());
        System.out.println("Student Age: " + student.getAge());
    }
}

```

```
System.out.println("The Student 1 Is : "+student);

// Creating another Student object using parameterized constructor
Student student2 = new Student(102, "Bob", 22);
System.out.println("The Student 2 Is : "+student2);

}

}
```

OUTPUT : -

```
PS E:\Yash Pandey\04_YashPandey_Codes(College)\008_Advanced-Java-Codes\006_JavaBeans> javac student_javabean/*.java
>>
PS E:\Yash Pandey\04_YashPandey_Codes(College)\008_Advanced-Java-Codes\006_JavaBeans> java student_javabean.StudentTest
>>

Student ID: 101
Student Name: Alice
Student Age: 20

The Student 1 Is : Student [ID=101, Name=Alice, Age=20]
The Student 2 Is : Student [ID=102, Name=Bob, Age=22]

PS E:\Yash Pandey\04_YashPandey_Codes(College)\008_Advanced-Java-Codes\006_JavaBeans> █
```

Lab Exercise-20. *(19/03/2025)*

Q :- WAP To Implement Employee JavaBean And Access It Using Another Java File ?

A **JavaBean** is a reusable, encapsulated Java class that follows specific conventions, making it easy to use in various applications. It must have a no-argument constructor, be serializable, and provide getter and setter methods for accessing private fields. JavaBeans are widely used in enterprise applications, particularly in Java EE, for managing data and enabling smooth communication between different application layers. They enhance modularity and maintainability, making them ideal for frameworks like Spring and JSP.

CODE :-

1. EmployeeBean.java File

```
package employee_javabean;
```

```
import java.io.Serializable;
```

```
public class Employee implements Serializable {
```

```
    private static final long serialVersionUID = 1L;
```

```
    private int empId;    private double salary;
```

```
    private String empName;
```

```
    public Employee() {}
```

```
    public int getEmpId() { return empId; }
```

```
    public void setEmpId(int empId) { this.empId = empId; }
```

```
    public String getEmpName() { return empName; }
```

```
    public void setEmpName(String empName) { this.empName = empName; }
```

```
public double getSalary() { return salary; }

public void setSalary(double salary) {
    if (salary > 0) { this.salary = salary; }
    else { System.out.println("Salary must be positive."); }
}

@Override
public String toString() {
    return "Employee [ID=" + empId + ", Name=" + empName + ", Salary=" + salary + "];"
}
}
```

2. EmployeeTest.java File

```
package employee_javabean;

public class EmployeeTest {

    public static void main(String[] args) {

        // Creating an Employee Bean using the no-arg constructor
        Employee emp = new Employee();

        // Setting properties using setter methods
        emp.setEmpId(1001);
        emp.setEmpName("John Doe");
        emp.setSalary(50000);

        // Getting properties using getter methods
        System.out.println("Employee ID: " + emp.getEmpId());
        System.out.println("Employee Name: " + emp.getEmpName());
    }
}
```

```
System.out.println("Employee Salary: $" + emp.getSalary());
```

```
System.out.println("The Employee Details Are : "+emp);
```

```
}
```

```
}
```

OUTPUT : -

```
PS E:\Yash Pandey\04_YashPandey_Codes(College)\008_Advanced-Java-Codes\006_JavaBeans> javac employee_javabean/*.java
```

```
>>
```

```
PS E:\Yash Pandey\04_YashPandey_Codes(College)\008_Advanced-Java-Codes\006_JavaBeans> java employee_javabean.EmployeeTest
```

```
>>
```

```
Employee ID: 1001
```

```
Employee Name: John Doe
```

```
Employee Salary: $50000.0
```

```
The Employee Details Are : Employee [ID=1001, Name=John Doe, Salary=50000.0]
```

```
PS E:\Yash Pandey\04_YashPandey_Codes(College)\008_Advanced-Java-Codes\006_JavaBeans> █
```

Lab Exercise-21. *(26/03/2025)*

Q :- Write a servlet using Web-Servlet to handle GET req. & display HTML response?

The HelloServlet program demonstrates a basic Java Servlet using annotation-based mapping with **@WebServlet("/hello")**. It extends `HttpServlet` and overrides the `doGet` method to handle GET requests. This example serves as a foundation for understanding servlet handling, response generation, and deployment in a Java web application.

The servlet sets the response content type to HTML and dynamically generates an HTML page using a `PrintWriter` object. When accessed via a web browser, it displays a "Hello, World!" message along with the creator's details.

CODE : -

1. HelloServlet.java File

```
import java.io.*;

import jakarta.servlet.*;

import jakarta.servlet.http.*;

import jakarta.servlet.annotation.*;

@WebServlet("/hello")

public class HelloServlet extends HttpServlet {

    private static final long serialVersionUID = 1L;

    protected void doGet(HttpServletRequest request , HttpServletResponse response)

        throws ServletException, IOException {

        response.setContentType("text/html");

        PrintWriter out = response.getWriter();

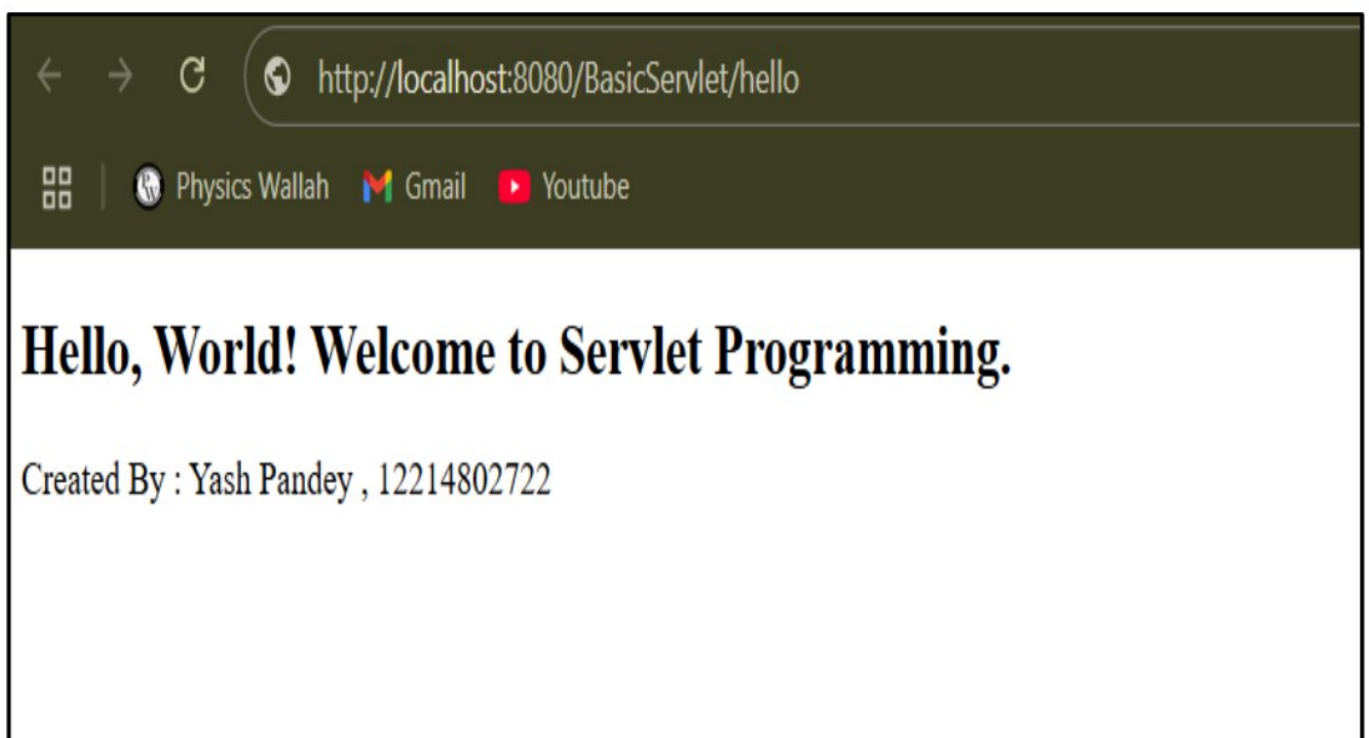
        out.println("<!DOCTYPE html><html lang='en'><head>");
```

```
out.println("<meta charset='UTF-8'><meta name='viewport'
            content='width=device-width, initial-scale=1.0'>");
out.println("<title>Document</title></head><body>");
out.println("<h2>Hello, World! Welcome to Servlet Programming.</h2>");
out.println("<p>Created By : Yash Pandey , 12214802722</p></body></html>");
}
}
```

2. Pom.xml File

```
<dependency>
    <groupId>jakarta.servlet</groupId>
    <artifactId>jakarta.servlet-api</artifactId>
    <version>5.0.0</version>
    <scope>provided</scope>
</dependency>
```

OUTPUT : -



Lab Exercise-22. (26/03/2025)

Q :- WAP using GenericServlet to handle requests and generate an HTML response ?

The **GenericServletExample** program demonstrates the use of GenericServlet, an abstract class that provides a simplified way to create servlets without directly implementing the HttpServlet class. It overrides the service method, which handles both GET and POST requests, setting the response content type to HTML and generating a dynamic webpage using a PrintWriter.

This servlet displays a "Hello, World!" message along with creator details, showcasing the fundamental structure of a generic servlet in Java web applications.

CODE : -

1. GenericServletExample.java File

```
import java.io.*;
import java.net.*;
import java.net.http.*;
import jakarta.servlet.*;

public class GenericServletExample extends GenericServlet {

    private static final long serialVersionUID = 1L;

    @Override

    public void service(ServletRequest request, ServletResponse response) throws

        ServletException, IOException {

        response.setContentType("text/html");

        PrintWriter out = response.getWriter();

        out.println("<html><body>");

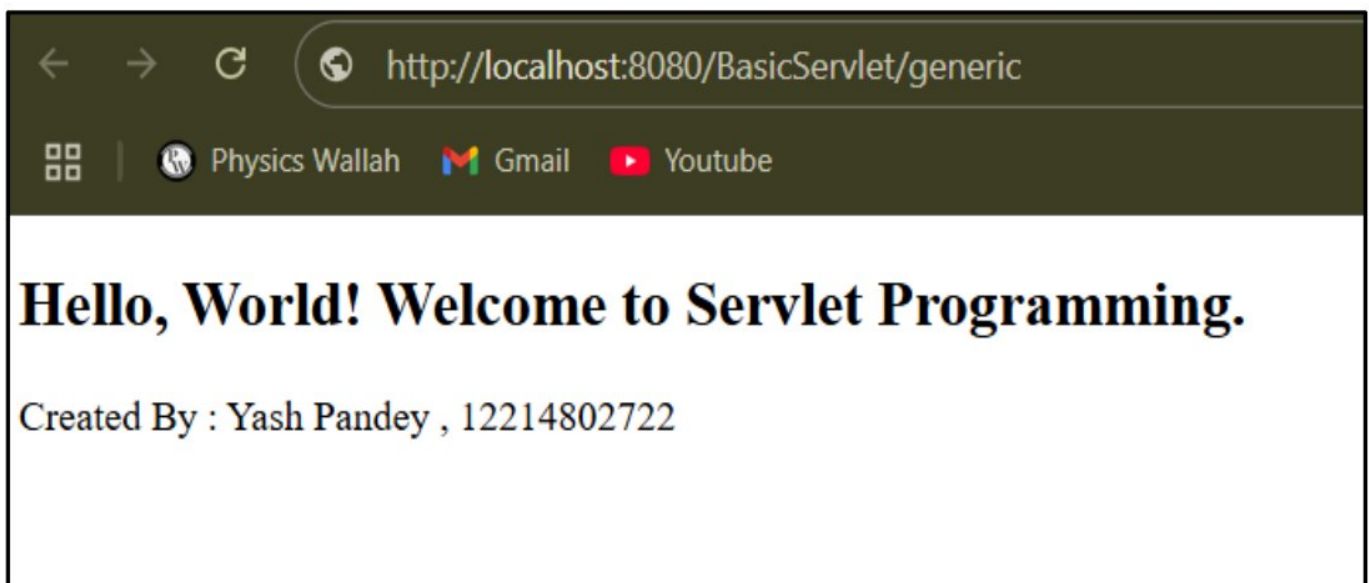
        out.println("<h2>Hello, World! Welcome to Servlet Programming.</h2>");
```

```
        out.println("<p>Created By : Yash Pandey , 12214802722</p>");
        out.println("</body></html>");
    }
}
```

2. web.xml File

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns="http://java.sun.com/xml/ns/javaee" version="3.0">
    <servlet>
        <servlet-name>GenericServletExample</servlet-name>
        <servlet-class>com.mycompany.mavenproject2.GenericServletExample</servlet-class>
    </servlet>
    <servlet-mapping>
        <servlet-name>GenericServletExample</servlet-name>
        <url-pattern>/hello</url-pattern>
    </servlet-mapping>
</web-app>
```

OUTPUT : -



Lab Exercise-23. *(26/03/2025)*

Q :- WAP to show lifecycle method like init(), service(), and destroy() of a GenericServlet ?

The **GenericServletExample** program illustrates the complete lifecycle of a GenericServlet by implementing the init(), service(), and destroy() methods. The constructor prints a message when the servlet is instantiated, while the init() method is called once to perform initialization.

The service() method handles incoming requests and generates an HTML response, and the destroy() method is executed before the servlet is removed from memory. This program provides a clear understanding of how a GenericServlet manages its lifecycle in a Java web application.

The **GenericServlet** class is an abstract class that simplifies servlet development by eliminating the need to implement HttpServlet when working with protocols other than HTTP. It provides a more flexible structure for creating servlets that can handle generic requests and responses. In this program, the GenericServletExample class overrides key lifecycle methods to demonstrate their execution order.

The constructor initializes the servlet, the init() method is executed once during servlet initialization, the service() method processes client requests dynamically, and the destroy() method ensures proper cleanup before the servlet is terminated. This showcases the fundamental working of a GenericServlet in a Java web environment.

CODE : -

```
import java.io.*;

import jakarta.servlet.*;

public class GenericServletExample extends GenericServlet {

    private static final long serialVersionUID = 1L;
```

```
public GenericServletExample() {  
    System.out.println("GenericServlet Constructor Called");  
}  
  
@Override  
public void init(ServletConfig config) throws ServletException {  
    super.init(config);  
    System.out.println("GenericServlet init() method called");  
}  
  
@Override  
public void service(ServletRequest request, ServletResponse response) throws  
    ServletException, IOException {  
    System.out.println("GenericServlet service() method called");  
    response.setContentType("text/html");  
    PrintWriter out = response.getWriter();  
    out.println("<html><body>");  
    out.println("<h2>This is a GenericServlet Example</h2>");  
    out.println("<p>Created By : Yash Pandey , 12214802722</p>");  
    out.println("</body></html>");  
}  
  
@Override  
public void destroy() {  
    System.out.println("GenericServlet destroy() method called");  
}  
}
```

OUTPUT On The Console : -

```
PS E:\Yash Pandey\04_YashPandey_Codes(College)\008_Advanced-Java-Codes\008_GenericServlet\SecondServletDemo-GenericServlet> javac -cp .; lib\* -d . src\com\mycompany\servlet\GenericServletExample1.java
```

```
PS E:\Yash Pandey\04_YashPandey_Codes(College)\008_Advanced-Java-Codes\008_GenericServlet\SecondServletDemo-GenericServlet> java -cp .; lib\* com.mycompany.servlet.GenericServletExample1
```

```
GenericServlet Constructor Called  
GenericServlet init() method called  
GenericServlet service() method called  
GenericServlet destroy() method called
```

```
PS E:\Yash Pandey\04_YashPandey_Codes(College)\008_Advanced-Java-Codes\008_GenericServlet\SecondServletDemo-GenericServlet> |
```

OUTPUT On The Browser : -



Lab Exercise-24. (26/03/2025)

Q :- WAP To show dynamically servlet resp. based on query parameters in the URL?

An **HTTP Servlet** is a Java class that extends `HttpServlet` to handle HTTP requests and responses in a web application. It runs on a **server (e.g., Tomcat)** and processes client requests sent via **GET, POST, or other HTTP methods**. The servlet takes input from the user (via URL parameters, form data, etc.), processes it, and generates a **dynamic response**, usually in **HTML format**. It follows the **request-response model**, making it a core component of Java web development. Methods like `doGet()`, `doPost()`, and `service()` define how it handles requests, while annotations like `@WebServlet` simplify deployment.

CODE :-

```
import java.io.*;
import java.net.*;
import jakarta.servlet.*;

@WebServlet(name = "NewServlet", urlPatterns = {"/NewServlet"})
public class NewServlet extends HttpServlet {
    protected void processRequest(HttpServletRequest request,
        HttpServletResponse response) throws ServletException, IOException {
        response.setContentType("text/html;charset=UTF-8");

        String name = request.getParameter("name");
        String age = request.getParameter("age");
        try (PrintWriter out = response.getWriter()) {
            out.println("<!DOCTYPE html>");
            out.println("<html><head><title>Dynamic Response</title></head><body>");
```

```

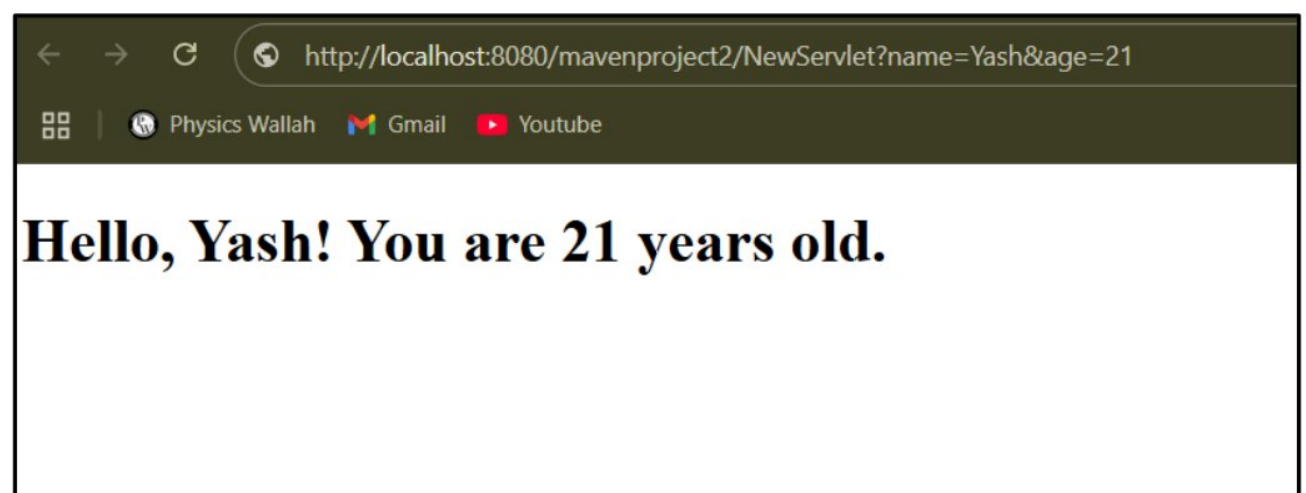
        if (name != null && age != null) {
            out.println("<h1>Hello, " + name + "! You are " + age + " years old.</h1>");
        } else {
            out.println("<h1>Missing parameters! Please provide name and age.</h1>");
        }
        out.println("</body></html>");
    }
}

@Override
protected void doGet(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
    processRequest(request, response);
}

@Override
protected void doPost(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
    processRequest(request, response);
}
}

```

OUTPUT :-



Lab Exercise-25. (26/03/2025)

Q :- WAP To show Login With HTTP Servlet ?

An **HTTP Servlet** is a Java class that extends `HttpServlet` to handle HTTP requests and responses in a web application. It runs on a **server (e.g., Tomcat)** and processes client requests sent via **GET, POST, or other HTTP methods**. The servlet takes input from the user (via URL parameters, form data, etc.), processes it, and generates a **dynamic response**, usually in **HTML format**. It follows the **request-response model**, making it a core component of Java web development. Methods like `doGet()`, `doPost()`, and `service()` define how it handles requests, while annotations like `@WebServlet` simplify deployment.

CODE : -

1. LoginServlet.java File

```
import java.io.*;
import java.net.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;

@WebServlet(name = "LoginServlet", urlPatterns = {"/LoginServlet"})

public class LoginServlet extends HttpServlet {

    @Override

    protected void doPost(HttpServletRequest request, HttpServletResponse response) throws
    ServletException, IOException {

        response.setContentType("text/html;charset=UTF-8");

        String username = request.getParameter("username");
        String password = request.getParameter("password");

        try (PrintWriter out = response.getWriter()) {
```

```

        out.println("<!DOCTYPE html>");
        out.println("<html><head><title>Login</title></head><body>");
        if ("admin".equals(username) && "password123".equals(password)) {
            out.println("<h2>Login Successful</h2>");
        } else {
            out.println("<h2>Invalid Credentials</h2>");
        }
        out.println("</body></html>");
    }
}

```

2. Login.html

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Login Servlet</title>
</head>
<body>
    <div class="container mt-4">
        <h1 style="margin-left: 220px">Login Form</h1>
        <form action="LoginServlet" method="post">
            <div class="col-6 mb-3">
                <label for="username" class="form-label">User Name</label>
                <input type="text" class="form-control" id="username" required />
            </div>
            <div class="col-6 mb-3">

```

```
<label for="Password" class="form-label">Password</label>
<input type="text" class="form-control" id="Password" required />
<div id="display" class="form-text">
    We'll never share your Username & Password with anyone else.
</div>
</div>
<button type="submit" class="btn btn-primary">Submit</button>
</form>
</div>
</body>
</html>
```

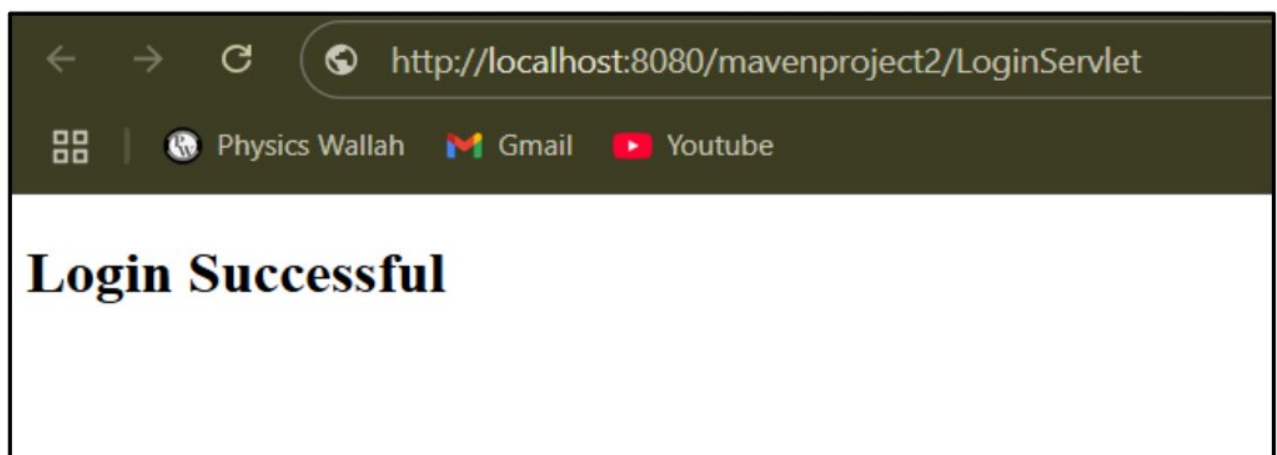
OUTPUT When Credentials Are Valid :-

Login Form

User Name

Password

We'll never share your Username & Password with anyone else.



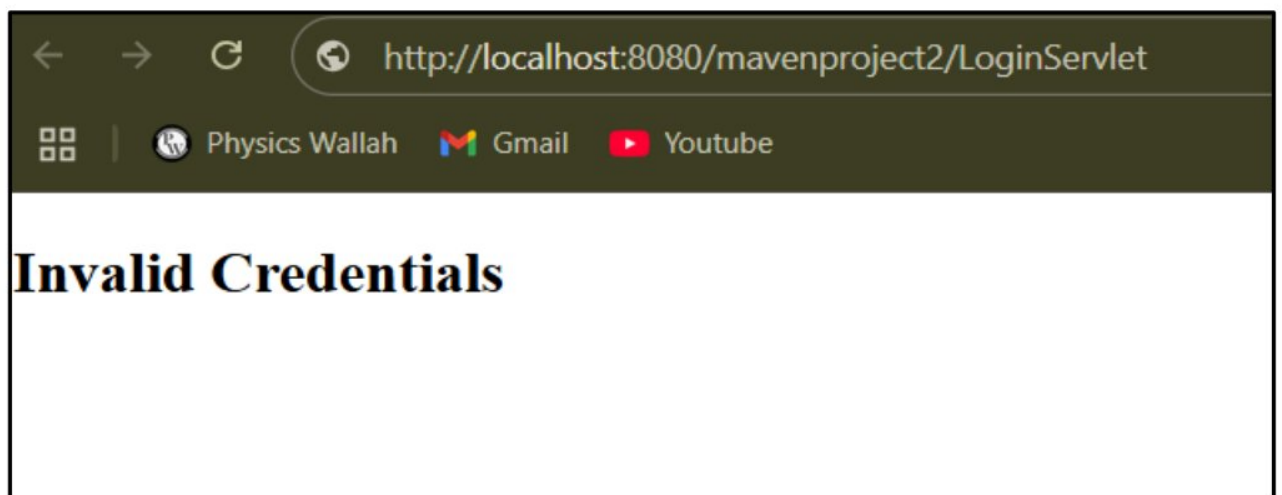
OUTPUT When Credentials Are Invalid :-

Login Form

User Name

Password

We'll never share your Username & Password with anyone else.



Lab Exercise-26. (26/03/2025)

Q :- WAP To Implement Cookie Servlet ?

A **Cookie Servlet** is a Java-based web component that manages HTTP cookies, allowing servers to store small pieces of data on a user's browser. In a typical implementation, a servlet creates cookies using the `HttpServletResponse` object and sends them to the client, where they are stored for a specified duration.

Later, another servlet can retrieve these cookies using the `HttpServletRequest` object, enabling session tracking, user authentication, and personalization. Cookies help maintain state in HTTP, which is inherently stateless, making them useful for features like remembering user preferences, login sessions, and shopping cart details in web applications.

CODE : -

1. CookieServlet.java File

```
import java.io.*;
import java.net.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;

public class CookieServlet extends HttpServlet {

    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        Cookie userCookie = new Cookie("username", "JohnDoe");
        userCookie.setMaxAge(60 * 60 * 24); // Expires in 1 day
        response.addCookie(userCookie);
    }
}
```

```
Cookie sessionCookie = new Cookie("sessionID", "123456");
response.addCookie(sessionCookie);

// Generating Response
out.println("<!DOCTYPE html>");
out.println("<html><head><title>Cookie Servlet</title></head><body>");
out.println("<h2>Cookies Created</h2>");
out.println("<p>Username: JohnDoe</p>");
out.println("<p>SessionID: 123456</p>");
out.println("<hr>");

// Retrieving and displaying all cookies
out.println("<h2>Cookies Information</h2>");
Cookie[] cookies = request.getCookies();
if (cookies != null) {
    for (Cookie cookie : cookies) {
        out.println("<p>Cookie Name: " + cookie.getName() + "</p>");
        out.println("<p>Cookie Value: " + cookie.getValue() + "</p>");
        out.println("<hr>");
    }
} else {
    out.println("<p>No cookies found</p>");
}
out.println("</body></html>");
}
```

2. GetCookieServlet.java File

```
import java.io.*;
import java.net.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;

public class GetCookiesServlet extends HttpServlet {

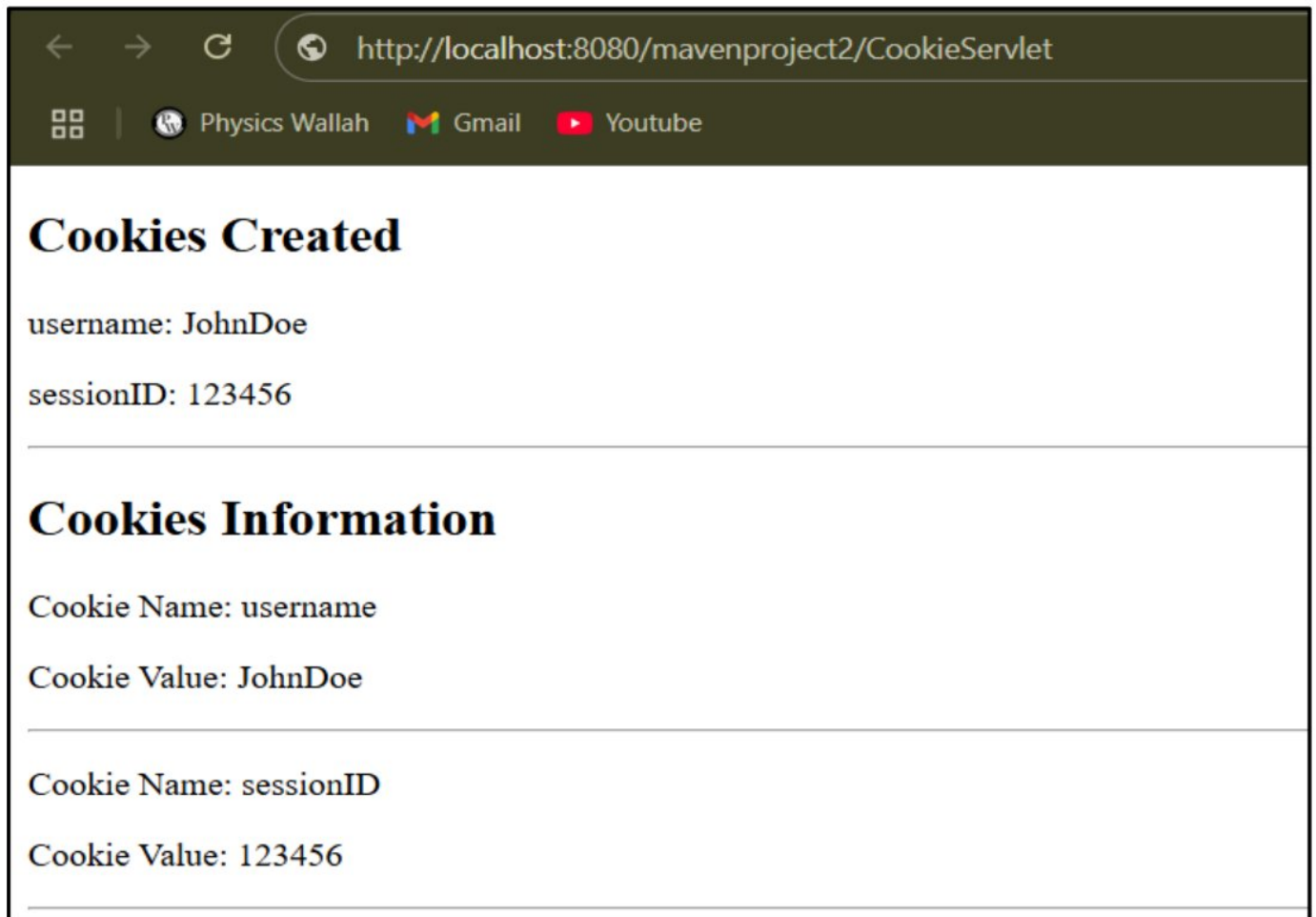
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        // Generating Response
        out.println("<!DOCTYPE html>");
        out.println("<html><head><title>Get Cookies Servlet</title></head><body>");
        out.println("<h2>Cookies Information</h2>");

        Cookie[] cookies = request.getCookies();
        if (cookies != null) {
            for (Cookie cookie : cookies) {
                out.println("<p>Cookie Name: " + cookie.getName() + "</p>");
                out.println("<p>Cookie Value: " + cookie.getValue() + "</p>");
                out.println("<hr>");
            }
        }
        else {
            out.println("<p>No cookies found</p>");
        }
        out.println("</body></html>");
    }
}
```

OUTPUT At '/mavenproject2/CookieServlet' :-



← → ↻ http://localhost:8080/mavenproject2/CookieServlet

☰ | 🌐 Physics Wallah 📧 Gmail 📺 Youtube

Cookies Created

username: JohnDoe

sessionID: 123456

Cookies Information

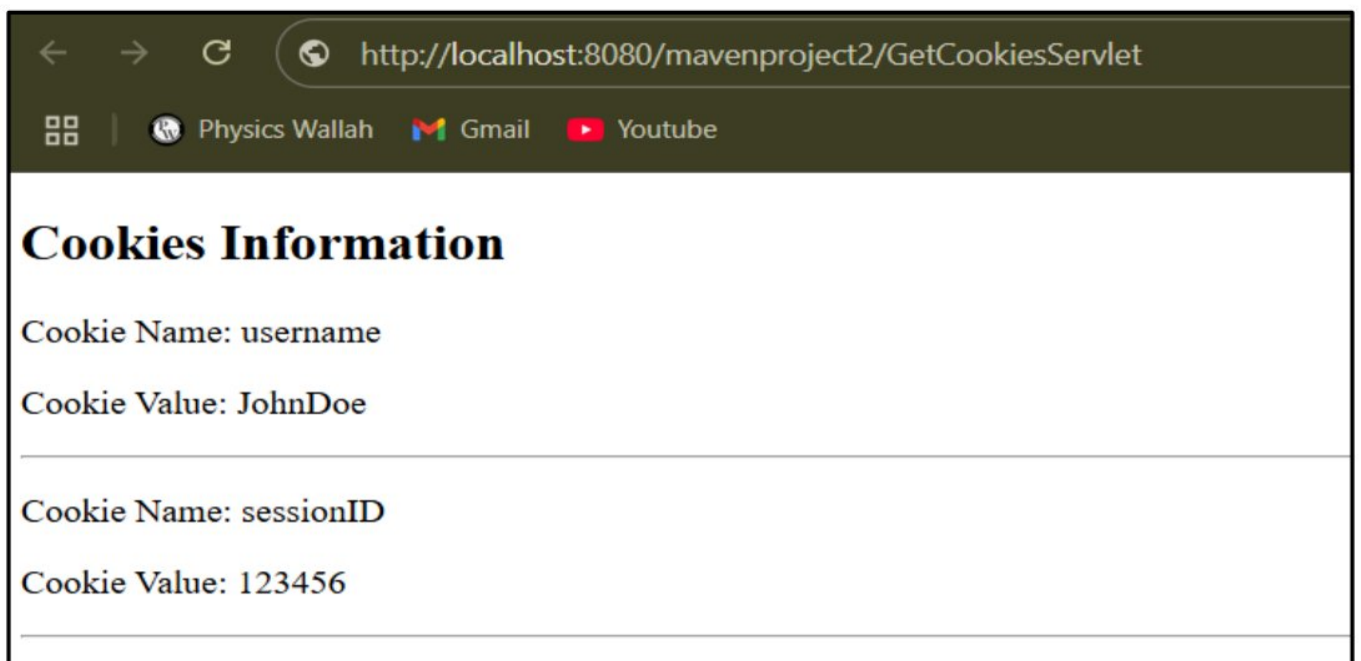
Cookie Name: username

Cookie Value: JohnDoe

Cookie Name: sessionID

Cookie Value: 123456

OUTPUT At '/mavenproject2/GetCookiesServlet' :-



← → ↻ http://localhost:8080/mavenproject2/GetCookiesServlet

☰ | 🌐 Physics Wallah 📧 Gmail 📺 Youtube

Cookies Information

Cookie Name: username

Cookie Value: JohnDoe

Cookie Name: sessionID

Cookie Value: 123456

Lab Exercise-27. (02/04/2025)

Q :- WAP To Represent An Human Skull ?

The Java code uses **basic 2D graphics** to visually mimic the appearance of a human skull. It starts by drawing a large white oval to represent the skull's main structure, followed by two black ovals positioned as eye sockets. A small triangle in the center acts as the nasal cavity, resembling the shape of a human nose hole. The mouth is illustrated using a curved arc to simulate a jawline, and rectangular white blocks are drawn beneath it to represent simplified teeth. Additionally, gray lines are used to depict cracks, adding a slightly realistic and rugged touch to the skull. Overall, the code creatively utilizes basic geometric shapes to form a stylized but recognizable skull.

The **Java's Swing and AWT libraries** to create and render graphics on a custom JPanel by overriding the paintComponent method. Inside this method, the Graphics object is cast to Graphics2D to take advantage of advanced rendering features like anti-aliasing, which smooths out the shapes for a cleaner visual output. Using fillOval, fillPolygon, drawArc, and fillRect methods .

CODE : -

```
import javax.swing.*;

import java.awt.*;

public class HumanSkullDrawing extends JPanel {

    @Override

    protected void paintComponent(Graphics g) {

        super.paintComponent(g);

        Graphics2D g2d = (Graphics2D) g;

        g2d.setRenderingHint(RenderingHints.KEY_ANTIALIASING,

            RenderingHints.VALUE_ANTIALIAS_ON);
```

// Skull outline

```
g2d.setColor(Color.WHITE);  
g2d.fillOval(100, 50, 200, 250);
```

// Eyes

```
g2d.setColor(Color.BLACK);  
g2d.fillOval(145, 120, 35, 50);  
g2d.fillOval(220, 120, 35, 50);
```

// Nose

```
int[] xPoints = {190, 210, 200};  
int[] yPoints = {180, 180, 220};  
g2d.fillPolygon(xPoints, yPoints, 3);
```

// Mouth

```
g2d.drawArc(150, 220, 100, 50, 0, -180);
```

// Teeth

```
g2d.setColor(Color.WHITE);  
g2d.fillRect(160, 230, 20, 30);  
g2d.fillRect(180, 230, 20, 30);  
g2d.fillRect(200, 230, 20, 30);  
g2d.fillRect(220, 230, 20, 30);  
g2d.fillRect(240, 230, 20, 30);
```

// Cracks

```
g2d.setColor(Color.GRAY);  
g2d.drawLine(160, 80, 220, 110);  
g2d.drawLine(230, 100, 170, 150);
```

```
}
```

```
public static void main(String[] args) {  
    JFrame frame = new JFrame("Human Skull Drawing");  
    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
    frame.setSize(400, 400);  
    frame.add(new HumanSkullDrawing());  
    frame.setVisible(true);  
}  
}
```

OUTPUT :-



Lab Exercise-28. (02/04/2025)

Q :- WAP To Implement Layout Managers?

In Java, **layout managers** are used to control the positioning and sizing of components within a container like a JPanel or JFrame. Instead of manually placing components using absolute coordinates, layout managers handle the layout automatically, making the interface more responsive and easier to maintain. Java provides several built-in layout managers such as **FlowLayout**, **BorderLayout**, **GridLayout**, **BoxLayout**, and **GridBagLayout**, each offering different strategies for arranging components. For example, FlowLayout places components in a row and wraps them as needed, while BorderLayout divides the container into five regions: North, South, East, West, and Center.

Using layout managers **promotes consistency in GUI design across different screen sizes and resolutions. Developers can combine multiple** layout managers by nesting panels, allowing for complex yet flexible designs. For instance, a top panel may use FlowLayout for a toolbar, the center panel might use GridLayout for a grid of buttons, and the bottom panel could use BorderLayout for action buttons.

The Different Layout Managers In Java Are :-

1. **FlowLayout**: Arranges components in a row, left to right, and wraps to the next line if needed—default layout for JPanel.
2. **BorderLayout**: Divides the container into five regions—North, South, East, West, and Center; each can hold one component.
3. **GridLayout**: Organizes components in a grid of equally sized rows and columns; all cells are of equal size.
4. **BoxLayout**: Arranges components in a single row (X_AXIS) or column (Y_AXIS) while respecting component size preferences.
5. **GridBagLayout**: A flexible and powerful layout that allows components to span multiple rows and columns with precise control over alignment and spacing.

CODE : -

```
import javax.swing.*;
```

```
import java.awt.*;
```

```
public class LayoutManager extends JFrame {
```

```
    public LayoutManager() {
```

```
        setTitle("Layout Manager Demo");
```

```
        setSize(500, 400);
```

```
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
```

```
        setLayout(new BorderLayout()); // Main layout
```

```
        // NORTH - FlowLayout Panel
```

```
        JPanel northPanel = new JPanel(new FlowLayout());
```

```
        northPanel.add(new JLabel("FlowLayout:"));
```

```
        northPanel.add(new JButton("One"));
```

```
        northPanel.add(new JButton("Two"));
```

```
        northPanel.add(new JButton("Three"));
```

```
        add(northPanel, BorderLayout.NORTH);
```

```
        // CENTER - GridLayout Panel
```

```
        JPanel centerPanel = new JPanel(new GridLayout(2, 2, 10, 10));
```

```
        centerPanel.setBorder(BorderFactory.createTitledBorder("GridLayout (2x2)"));
```

```
        centerPanel.add(new JButton("A"));
```

```
        centerPanel.add(new JButton("B"));
```

```
        centerPanel.add(new JButton("C"));
```

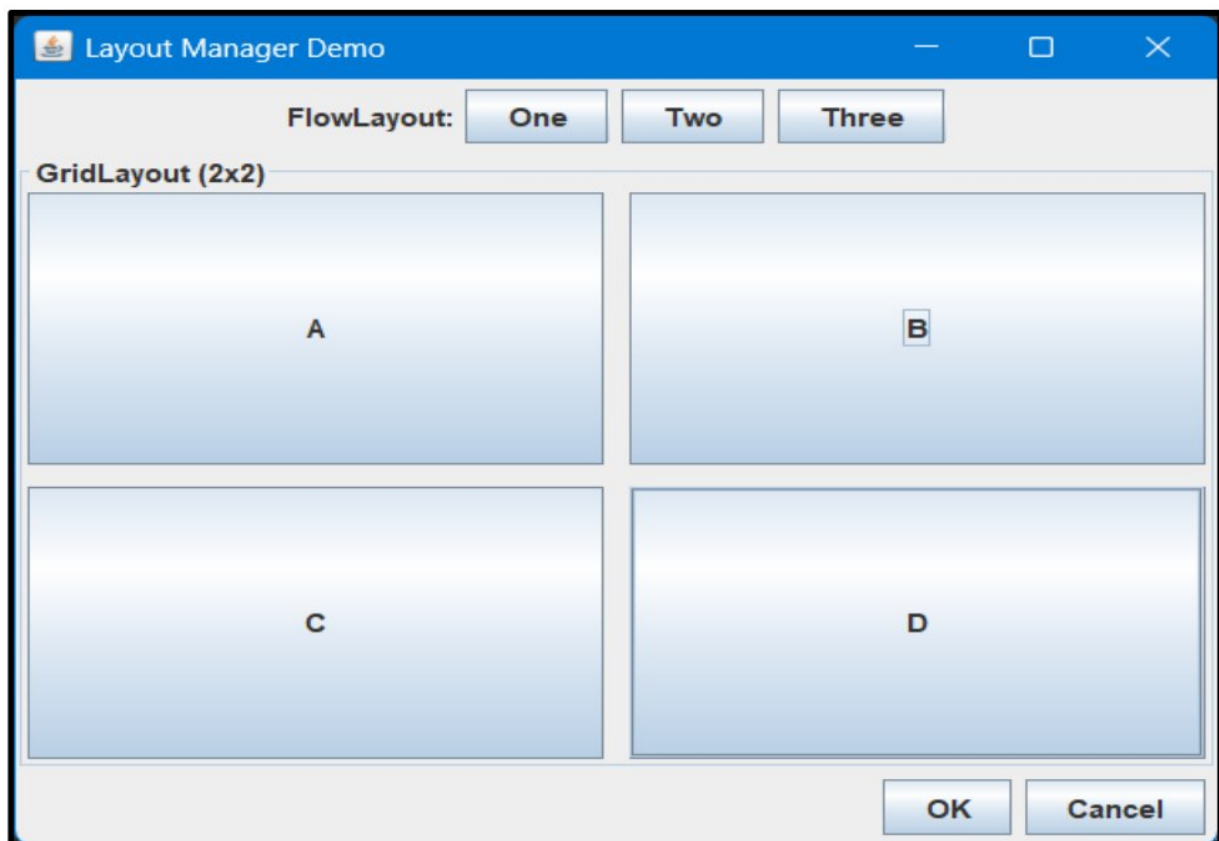
```
        centerPanel.add(new JButton("D"));
```

```
        add(centerPanel, BorderLayout.CENTER);
```

```
// SOUTH - FlowLayout with Right Alignment
JPanel southPanel = new JPanel(new FlowLayout(FlowLayout.RIGHT));
southPanel.add(new JButton("OK"));
southPanel.add(new JButton("Cancel"));
add(southPanel, BorderLayout.SOUTH);
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> {
        LayoutManager demo = new LayoutManager();
        demo.setVisible(true);
    });
}
}
```

OUTPUT :-



Lab Exercise-29. (02/04/2025)

Q :- WAP To Show Adapter Class In Java?

In Java, an **Adapter class** is a convenient way to handle event listeners by providing default (empty) implementations of all methods in an interface. This is especially useful for interfaces with multiple methods, like `MouseListener`, where you might only need one or two. Instead of implementing the entire interface and writing empty bodies for unused methods, you can simply extend an adapter class like `MouseAdapter`, `KeyAdapter`, or `WindowAdapter` and override only the required methods. This approach leads to cleaner, more readable code in GUI applications or event-driven programming.

CODE : -

```
import javax.swing.*;

import java.awt.*;

import java.awt.event.*;

public class AdapterClass extends JFrame {

    JLabel label;

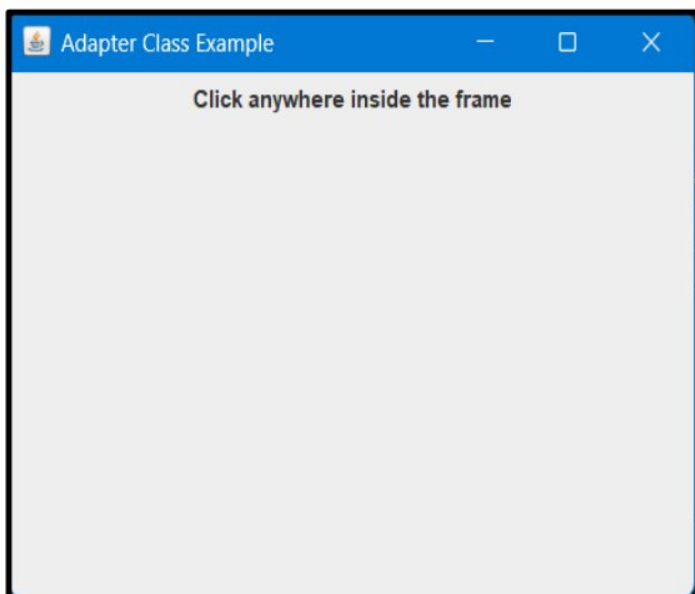
    public AdapterClass() {
        setTitle("Adapter Class Example");
        setSize(400, 300);
        setLayout(new FlowLayout());
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        label = new JLabel("Click anywhere inside the frame");
        add(label);
    }
}
```

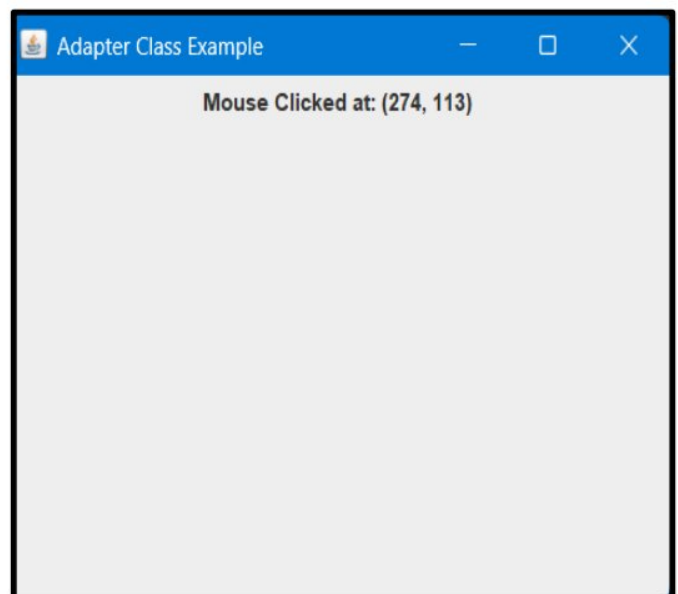
```
// Using MouseAdapter (Adapter class) instead of implementing MouseListener directly
addMouseListener(new MouseAdapter() {
    @Override
    public void mouseClicked(MouseEvent e) {
        label.setText("Mouse Clicked at: (" + e.getX() + ", " + e.getY() + ")");
    }
});

public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> {
        AdapterClass demo = new AdapterClass();
        demo.setVisible(true);
    });
}
```

OUTPUT :-



Before Click Of Mouse



After Click Of Mouse

Lab Exercise-30. (02/04/2025)

Q :- WAP To Implement An Chat Application?

This **Java-based chat application** is a simple console-driven client-server model where multiple clients can connect and communicate in real time. The server (ChatServer) listens for incoming client connections using sockets and spawns a separate thread for each client using the ClientHandler class.

It broadcasts received messages to all connected clients except the sender, creating a group chat effect. On the other side, the client (ChatClient) connects to the server, sends the user's name, and allows real-time message sending and receiving using separate threads — one for sending input from the user and another for listening to server messages. This application demonstrates core networking concepts in Java using Socket, ServerSocket and basic I/O streams.

CODE :-

1. ChatServer.java

```
import java.io.*;
import java.net.*;
import java.util.*;

public class ChatServer {

    private static final List<ClientHandler> clients = new ArrayList<>();

    public static void main(String[] args) {

        int port = 12345;

        try (ServerSocket serverSocket = new ServerSocket(port)) {

            System.out.println("Chat Server started on port " + port);

            while (true) {

                Socket clientSocket = serverSocket.accept();

                System.out.println("New client connected: " + clientSocket.getInetAddress());

                ClientHandler clientHandler = new ClientHandler(clientSocket);
```

```

        clients.add(clientHandler);
        new Thread(clientHandler).start();
    }
} catch (IOException e) { e.printStackTrace(); }
}

public static void broadcastMessage(String message, ClientHandler sender) {
    for (ClientHandler client : clients) {
        if (client != sender) client.sendMessage(message);
    }
}

private static class ClientHandler implements Runnable {
    private Socket socket;
    private PrintWriter out;
    private BufferedReader in;
    private String clientName;
    public ClientHandler(Socket socket) {
        this.socket = socket;
        try {
            in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
            out = new PrintWriter(socket.getOutputStream(), true);
            out.println("Enter your name: ");
            clientName = in.readLine();
            out.println("Welcome to the chat, " + clientName + "!");
        } catch (IOException e) { e.printStackTrace(); }
    }

    public void sendMessage(String message) { out.println(message); }

    @Override
    public void run() {
        String message;

```

```

try {
    while ((message = in.readLine()) != null) {
        if (message.equalsIgnoreCase("exit")) break;
        System.out.println(clientName + ": " + message);
        ChatServer.broadcastMessage(clientName + ": " + message, this);
    }
} catch (IOException e) { e.printStackTrace(); }
finally {
    try {
        clients.remove(this);
        socket.close();
        System.out.println(clientName + " has disconnected.");
    } catch (IOException e) { e.printStackTrace(); }
}
}
}
}

```

2. ChatClient.java

```

import java.io.*;
import java.net.*;

public class ChatClient {

    private static PrintWriter out; private static BufferedReader in, userInput;

    public static void main(String[] args) {

        String serverAddress = "localhost"; int port = 12345;
        try (Socket socket = new Socket(serverAddress, port)) {
            in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
            out = new PrintWriter(socket.getOutputStream(), true);
            userInput = new BufferedReader(new InputStreamReader(System.in));

```

```

        out.println(userInput.readLine());
        new Thread(new ServerListener()).start();
        String message;
        while (true) {
            message = userInput.readLine();
            out.println(message);
        }
    } catch (IOException e) { e.printStackTrace(); }
}

private static class ServerListener implements Runnable {
    public void run() {
        String message;
        try {
            while ((message = in.readLine()) != null) System.out.println(message)
        } catch (IOException e) { e.printStackTrace(); }
    }
}
}

```

OUTPUT :-

```

[yash@nixos:~/Repo/JAVA/Week8]$ java ChatClient
Enter your name:  Yash Pandey
Welcome to the chat, Yash Pandey!
Hello

```

```

[yash@nixos:~/Repo/JAVA/Week8]$ java ChatServer
Chat Server started on port 12345
New client connected: /127.0.0.1
Yash Pandey: Hello

```

Lab Exercise-31. (09/04/2025)

Q :- WAP To Implement An User Login Using Java Server Pages and JavaBeans?

The **JavaServer Pages (JSP)** is a server-side technology that allows the creation of dynamic, platform-independent web content using Java. It enables developers to embed Java code directly into HTML pages using special tags, making it easier to build interactive web applications. JSP is compiled into servlets by the server and executed, allowing for efficient handling of requests and responses. It supports the use of various implicit objects like request, response, session, and application, which simplify interaction with web elements. JSP is particularly useful for separating the presentation layer from the business logic in a Java web application.

A **JavaBean** is a reusable software component in Java that follows specific conventions such as having a no-argument constructor, providing getter and setter methods to access private properties, and being serializable. JavaBeans are used to encapsulate data and logic into a single object, making it easier to manage and reuse across applications. In the context of JSP and web development, JavaBeans are often used to handle form data, perform calculations, or interact with databases, while maintaining a clean separation between the logic and presentation layers. This makes them ideal for use in Model-View-Controller (MVC) architecture.

CODE : -

1. LoginBean.java File

```
public class LoginBean {  
    private String username; private String password;  
    public String getUsername() { return username; }  
    public void setUsername(String username) {  
        this.username = username;  
    }  
}
```

```
public String getPassword() {  
    return password;  
}  
  
public void setPassword(String password) {  
    this.password = password;  
}  
  
public boolean validate() {  
    return "admin".equals(username) && "admin123".equals(password);  
}  
}
```

2. Login.jsp File

```
<%@ page language="java" contentType="text/html; charset=UTF-8" %>  
  
<html>  
  
<head>  
    <title>Login Page</title>  
</head>  
  
<body>  
    <h2>User Login</h2>  
    <form action="result.jsp" method="post">  
        Username: <input type="text" name="username" /><br/><br/>  
        Password: <input type="password" name="password" /><br/><br/>  
        <input type="submit" value="Login" />  
    </form>  
</body>  
</html>
```

3. Result.jsp File

```
<%@ page import="beans.LoginBean" %>

<jsp:useBean id="login" class="beans.LoginBean" scope="request" />

<jsp:setProperty name="login" property="username" param="username" />

<jsp:setProperty name="login" property="password" param="password" />


<html>

<head>

    <title>Login Result</title>

</head>

<body>

    <%

        if (login.validate()) {

    %>

        <h2>Welcome, <%= login.getUsername() %>!</h2>

    <%

        } else {

    %>

        <h2>Invalid credentials. Please try again.</h2>

        <a href="login.jsp">Go back to login</a>

    <%

        }

    %>

</body>

</html>
```

OUTPUT (Before Submit Of Form):-

User Login Using JSP

UserName

Password

We'll never share your email with anyone else.

OUTPUT (After Submit Of Form):-

Welcome, YashPandey29!

[Go back to login](#)

Lab Exercise-32. (09/04/2025)

Q :- WAP To Implement An Simple Interest Calculator Using JSP and JavaBeans?

The **JavaServer Pages (JSP)** is a server-side technology that allows the creation of dynamic, platform-independent web content using Java. It enables developers to embed Java code directly into HTML pages using special tags, making it easier to build interactive web applications. JSP is compiled into servlets by the server and executed, allowing for efficient handling of requests and responses. It supports the use of various implicit objects like request, response, session, and application, which simplify interaction with web elements. JSP is particularly useful for separating the presentation layer from the business logic in a Java web application.

A **JavaBean** is a reusable software component in Java that follows specific conventions such as having a no-argument constructor, providing getter and setter methods to access private properties, and being serializable. JavaBeans are used to encapsulate data and logic into a single object, making it easier to manage and reuse across applications. In the context of JSP and web development, JavaBeans are often used to handle form data, perform calculations, or interact with databases, while maintaining a clean separation between the logic and presentation layers. This makes them ideal for use in Model-View-Controller (MVC) architecture.

CODE :-

1. InterestBean.java File

```
public class InterestBean {  
    private double principal;  
    private double rate;  
    private double time;  
    public double getPrincipal() {    return principal;    }  
}
```

```
public void setPrincipal(double principal) {  
    this.principal = principal;  
}  
  
public double getRate() {    return rate;        }  
  
public void setRate(double rate) {  
    this.rate = rate;  
}  
  
public double getTime() {    return time;        }  
  
public void setTime(double time) {  
    this.time = time;  
}  
  
public double calculateInterest() {  
    return (principal * rate * time) / 100;  
}  
}
```

2. Interest.jsp File

```
<%@ page language="java" contentType="text/html; charset=UTF-8" %>  
<html>  
    <head>  
        <title>Simple Interest Calculator</title>  
    </head>  
    <body>  
        <h2>Enter Details to Calculate Simple Interest</h2>  
        <form action="result.jsp" method="post">  
            Principal: <input type="text" name="principal" /><br /><br />  
            Rate (%): <input type="text" name="rate" /><br /><br />  
            Time (years): <input type="text" name="time" /><br /><br />
```

```
        <input type="submit" value="Calculate Interest" />
    </form>
</body>
</html>
```

3. Result.jsp File

```
<%@ page import="beans.InterestBean" %>

<jsp:useBean id="interest" class="beans.InterestBean" scope="request" />
<jsp:setProperty name="interest" property="principal" param="principal" />
<jsp:setProperty name="interest" property="rate" param="rate" />
<jsp:setProperty name="interest" property="time" param="time" />

<html>
    <head>
        <title>Interest Result</title>
    </head>
    <body>
        <h2>Calculation Result</h2>

        <% double si = interest.calculateInterest(); %>
        <p>Principal: <%= interest.getPrincipal() %></p>
        <p>Rate: <%= interest.getRate() %>%</p>
        <p>Time: <%= interest.getTime() %> years</p>
        <hr />
        <p><strong>Simple Interest: <%= si %></strong></p>
        <br />
        <a href="interest.jsp">Calculate Again</a>
    </body>
</html>
```

OUTPUT (Before Submit Of Form):-

Enter Details to Calculate Simple Interest

Principal

Rate (%)

Time (years)

[Calculate Interest](#)

OUTPUT (After Submit Of Form):-

Calculation Result

Principal: 1,00,000

Rate: 10%

Time: 5 years

Simple Interest: 50,000

[Calculate Again](#)

Lab Exercise-33. (09/04/2025)

Q :- WAP To Implement An Marks & Grade Calculator Using JSP and JavaBeans?

The **JavaServer Pages (JSP)** is a server-side technology that allows the creation of dynamic, platform-independent web content using Java. It enables developers to embed Java code directly into HTML pages using special tags, making it easier to build interactive web applications. JSP is compiled into servlets by the server and executed, allowing for efficient handling of requests and responses. It supports the use of various implicit objects like request, response, session, and application, which simplify interaction with web elements. JSP is particularly useful for separating the presentation layer from the business logic in a Java web application.

A **JavaBean** is a reusable software component in Java that follows specific conventions such as having a no-argument constructor, providing getter and setter methods to access private properties, and being serializable. JavaBeans are used to encapsulate data and logic into a single object, making it easier to manage and reuse across applications. In the context of JSP and web development, JavaBeans are often used to handle form data, perform calculations, or interact with databases, while maintaining a clean separation between the logic and presentation layers. This makes them ideal for use in Model-View-Controller (MVC) architecture.

CODE :-

1. MarksBean.java File

```
public class MarksBean {  
    private int mark1, mark2, mark3, mark4, mark5 , mark6;  
    public int getMark1() { return mark1; }  
    public void setMark1(int mark1) { this.mark1 = mark1; }
```

```
public int getMark2() { return mark2; }

public void setMark2(int mark2) { this.mark2 = mark2; }

public int getMark3() { return mark3; }

public void setMark3(int mark3) { this.mark3 = mark3; }

public int getMark4() { return mark4; }

public void setMark4(int mark4) { this.mark4 = mark4; }

public int getMark5() { return mark5; }

public void setMark5(int mark5) { this.mark5 = mark5; }

public int getMark6() { return mark6; }

public void setMark6(int mark6) { this.mark6 = mark6; }

public int getTotal() {
    return mark1 + mark2 + mark3 + mark4 + mark5 + mark6;
}

public double getAverage() {
    return getTotal() / 5.0;
}

public String getGrade() {
    double avg = getAverage();
    if (avg >= 90) return "A";
    else if (avg >= 75) return "B";
    else if (avg >= 60) return "C";
    else if (avg >= 40) return "D";
    else return "F";
}
}
```

2. Marks.jsp File

```
<%@ page language="java" contentType="text/html; charset=UTF-8" %>
<html>
  <head>  <title>Student Marks Entry</title> </head>
  <body>
    <h2>Enter Marks for 6 Subjects</h2>
    <form action="result.jsp" method="post">
      Advanced Java Progg.: <input type="text" name="mark1" /><br /><br />
      Progg. In Python: <input type="text" name="mark2" /><br /><br />
      Web Tech.: <input type="text" name="mark3" /><br /><br />
      Artificial Intelligence: <input type="text" name="mark4" /><br /><br />
      Statistics: <input type="text" name="mark5" /><br /><br />
      Principle Of Management: <input type="text" name="mark6" /><br /><br />
      <input type="submit" value="Calculate" />
    </form>
  </body>
</html>
```

3. Result.jsp File

```
<%@ page import="beans.MarksBean" %>
<jsp:useBean id="student" class="beans.MarksBean" scope="request" />
<jsp:setProperty name="student" property="mark1" param="mark1" />
<jsp:setProperty name="student" property="mark2" param="mark2" />
<jsp:setProperty name="student" property="mark3" param="mark3" />
<jsp:setProperty name="student" property="mark4" param="mark4" />
<jsp:setProperty name="student" property="mark5" param="mark5" />
<jsp:setProperty name="student" property="mark5" param="mark6" />
<html>
  <head>  <title>Student Result</title> </head>
```

```
<body>

<p>Total Marks: <%= student.getTotal() %></p>

<p>Average Marks: <%= student.getAverage() %></p>

<p>Grade: <strong> <%= student.getGrade() %> </strong></p>

<a href="marks.jsp">Enter Again</a>

</body>

</html>
```

OUTPUT (Before Submit Of Form):-

Enter Marks for 6 Subjects

Advanced Java Progg.	Progg. In Python
90	95
Web Tech.	Artificial Intelligence
95	90
Statistics	Principle Of Management
90	90

Calculate

OUTPUT (After Submit Of Form):-

Result Summary

Total Marks: **550**

Average Marks: **91.67**

Grade: **A**

Enter Again